

STM32 单片机

目录

第 1 章 电源控制.....	3
1.1 电源框图.....	3
1.2 上电复位和掉电复位.....	4
1.3 可编程电压检测器（PVD）.....	5
1.4 低功耗.....	6
1.4.1 睡眠模式.....	6
1.4.2 停机（停止）模式.....	7
1.4.3 待机模式.....	8
1.5 低功耗案例 1：睡眠模式.....	8
1.5.1 需求描述.....	8
1.5.2 软件设计（寄存器）.....	8
1.5.3 软件设计（HAL 库）.....	14
1.6 低功耗案例 2：停机模式.....	16
1.6.1 需求描述.....	16
1.6.2 硬件电路设计.....	16
1.6.3 软件设计（寄存器）.....	16
1.6.4 软件设计（HAL 库）.....	22
1.7 低功耗案例 3：待机模式.....	24
1.7.1 需求描述.....	24
1.7.2 软件设计（寄存器）.....	24
1.7.3 软件设计（HAL 库）.....	26
第 2 章 RTC（实时时钟）.....	28
2.1 RTC 简介.....	28
2.2 掉电继续运行.....	28
2.3 RTC 功能框图.....	29
第 3 章 备份寄存器（BKP）.....	32
3.1 备份寄存器介绍.....	32
3.1.1 备份寄存器简介.....	32
3.1.2 侵入检测功能.....	32
3.1.3 RTC 校准.....	32
3.1.4 备份数据寄存器.....	32
3.1.5 其他寄存器.....	33
3.2 备份寄存器案例：读写备份寄存器.....	33
3.2.1 需求描述.....	33
3.2.2 硬件电路设计.....	33
3.2.3 软件设计（寄存器）.....	33
3.2.4 软件设计（HAL 库）.....	36
第 4 章 RTC 实验.....	39
4.1 RTC 案例 1：使用闹钟唤醒待机模式.....	39
4.1.1 需求描述.....	39

4.1.2 软件设计（寄存器）	39
4.1.3 软件设计（HAL 库）	42
4.2 RTC 案例 2：实时时钟（掉电不丢失）	44
4.2.1 需求描述	44
4.2.2 软件设计（寄存器）	44
4.2.3 软件设计（HAL 库）	48
第 5 章 看门狗	50
5.1 什么是看门狗	50
5.2 STM32 的看门狗	50
5.2.1 独立看门狗	50
5.2.2 窗口看门狗	52
5.2.3 独立看门狗和窗口看门口的对比	55
5.3 独立看门狗案例	55
5.3.1 需求描述	56
5.3.2 软件设计（寄存器）	56
5.3.3 软件设计（HAL 库）	58
第 6 章 附录 1：HAL 库代码分析	60
6.1 HAL 库目录结构	60
6.2 使用 STM32CubeMX 创建项目	61
6.3 项目中主要文件的作用	61
6.3.1 startup_stm32f103xe.s	62
6.3.2 system_stm32f1xx.c	62
6.3.3 stm32f1xx_it.c	63
6.3.4 stm32f1xx_hal_conf.h	63
6.3.5 stm32f1xx_hal.c	64
6.4 main 函数的执行	64
6.4.1 HAL_Init()	65
6.4.2 SystemClock_Config()	65
6.4.3 MX_GPIO_Init	66
6.4.4 MX_USART1_UART_Init	67
6.5 HAL 编程模式	68
第 7 章 附录 2：Keil MDK 程序编译过程	70
7.1 普通 C 源文件到可执行的过程	70
7.2 Keil MDK 编译过程	70
7.3 程序组成	71
7.4 程序的静止状态和运行状态	72

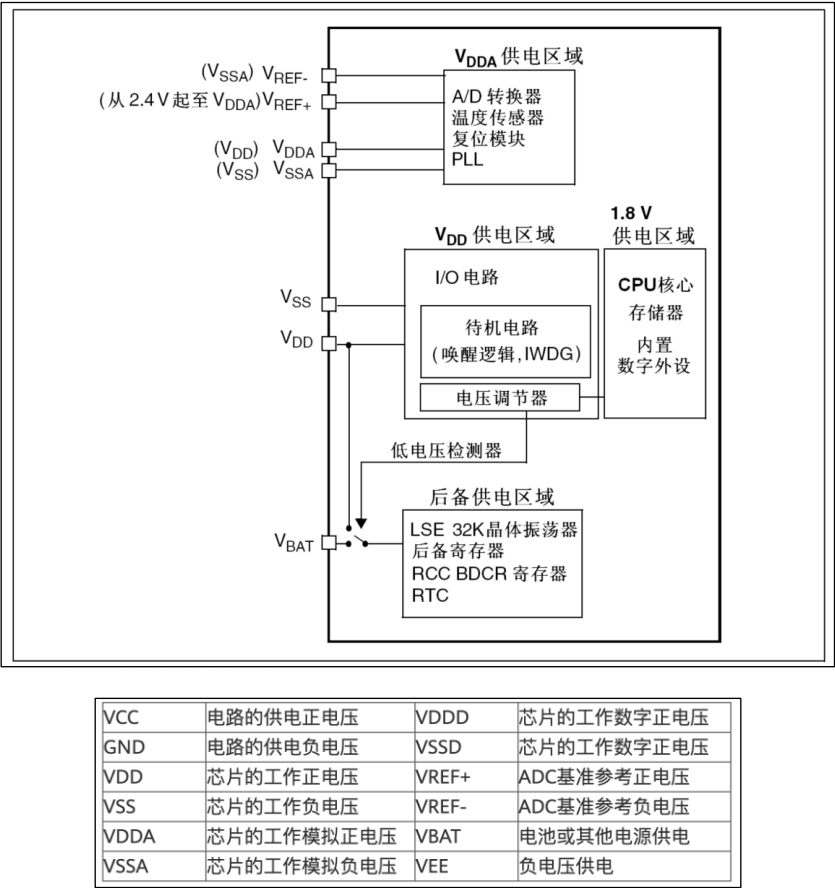
第 1 章 电源控制

电源控制（PWR： Power Control）。电源对电子设备来说非常重要，它是保证系统稳定运行的基础。在保证系统能稳定运行的同时，对嵌入式设备一般又有低功耗的需求。

在一些应用场合中，对电子设备的功耗要求非常苛刻，如某些传感器信息采集设备，仅靠小型的电池提供电源，要求工作长达数年之久，且期间不需要任何维护；由于智慧穿戴设备的小型化要求，电池体积不能太大导致容量也比较小，所以也很有必要从控制功耗入手，提高设备的续行时间。

鉴于低功耗的需求，STM32 有专门的电源管理外设监控电源并管理设备的运行式，确保系统正常运行，并尽量降低器件的功耗。

1.1 电源框图



1) V_{DDA} 供电区域

主要负责模拟部分的供电。为了提高转换的精确度，ADC 使用一个独立的电源供电，过滤和屏蔽来自印刷电路板上的毛刺干扰。

主要给 A/D 转换器，温度传感器，复位模块，PLL 等供电。 V_{DDA} 是正极， V_{SSA} 是负极。 V_{REF+} 和 V_{REF-} 是 A/D 转换器的参考电压， V_{REF-} 必须接到 V_{SSA} 上。对引脚比较少的芯片，没有 V_{REF+} 和 V_{REF-} 引脚，他们在芯片内部与 ADC 的电源（ V_{DDA} ）和地（ V_{SSA} ）相联。

2) V_{DD} 供电区域

V_{DD} 是数字电的正极， V_{SS} 是数字电的负极，电压 3.3V。给 I/O 电路，待机电路和电压调节器供电。

3) 1.8V 低电压区域

从这里可以看出，CPU 核心，内部存储器，内置的数字外设都是工作在 1.8V 的电压。1.8V 的低电压是由从电压调节器得到的。

4) 后备供电区域

V_{BAT} 引脚会接电池和其他电源，当 V_{DD} 断电时，可以保存备份寄存器的内容和给 RTC 供电。

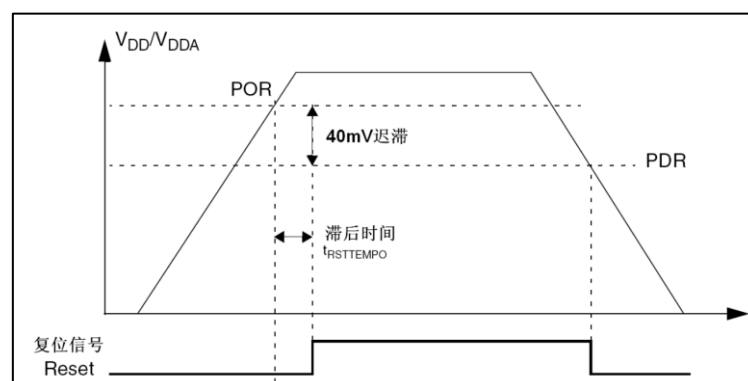
V_{BAT} 脚也为 RTC、LSE 振荡器和 PC13 至 PC15 供电，这保证当主要电源被切断时 RTC 能继续工作。切换到 V_{BAT} 供电由复位模块中的掉电复位功能控制。如果应用中没有使用外部电池， V_{BAT} 必须连接到 V_{DD} 引脚上。

5) 电压调节器

复位后，电压调节器总是工作使能的。有 3 种不同的工作模式：

- （1）运转模式：调节器以正常功耗模式提供 1.8V 电源(内核，内存和外设)。
- （2）停止模式：调节器以低功耗模式提供 1.8V 电源，用于**保存寄存器和 SRAM 的内容**。
- （3）待机模式：调节器停止供电。除了备用电路和备份域外，**寄存器和 SRAM 的内容全部丢失**

1.2 上电复位和掉电复位



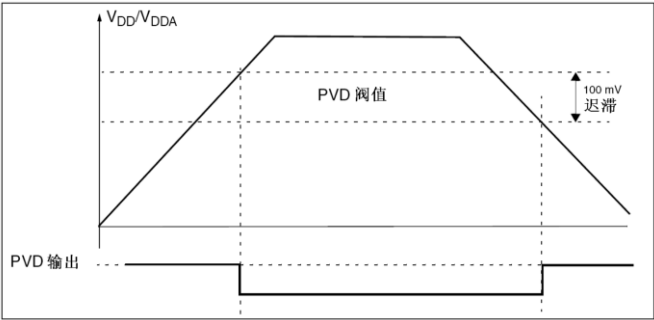
当 V_{DD}/V_{DDA} 低于指定的限位电压 V_{POR}/V_{PDR} 时，系统保持为复位状态，而无需外部复位电路。

复位和解除复位有一个 40mV 的迟滞电压。当电压大于 V_{POR} 时解除复位，当电压小于 V_{PDR} 时进入复位。设置 2 个阈值的作用是当电压在附近抖动的时候防止频繁的复位和解除复位。

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
$V_{POR/PDR}$	Power on/power down reset threshold	Falling edge	1.8 ⁽¹⁾	1.88	1.96	V
		Rising edge	1.84	1.92	2.0	
$V_{PDRhyst}^{(2)}$	PDR hysteresis 迟滞电压	-	-	40	-	mV
$T_{RSTTEMPO}^{(2)}$	Reset temporization 复位持续时间	-	1	2.5	4.5	ms

1.3 可编程电压检测器（PVD）

PVD（Programmable voltage detector）。PVD 也是监测 V_{DD}/V_{DDA} 的供电电压，它的阈值电压可以编程。



用户可以利用 PVD 对 V_{DD} 电压与电源控制寄存器(PWR_CR)中的 PLS[2:0]位进行比较来监控电源。当电压达到阈值的时候，不是产生复位，而是产生中断。可以在中断响应函数内做一些紧急处理。

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
V_{PVD}	Programmable voltage detector level selection	PLS[2:0]=000 (rising edge)	2.1	2.18	2.26	V
		PLS[2:0]=000 (falling edge)	2	2.08	2.16	
		PLS[2:0]=001 (rising edge)	2.19	2.28	2.37	
		PLS[2:0]=001 (falling edge)	2.09	2.18	2.27	
		PLS[2:0]=010 (rising edge)	2.28	2.38	2.48	
		PLS[2:0]=010 (falling edge)	2.18	2.28	2.38	
		PLS[2:0]=011 (rising edge)	2.38	2.48	2.58	
		PLS[2:0]=011 (falling edge)	2.28	2.38	2.48	
		PLS[2:0]=100 (rising edge)	2.47	2.58	2.69	
		PLS[2:0]=100 (falling edge)	2.37	2.48	2.59	
		PLS[2:0]=101 (rising edge)	2.57	2.68	2.79	
		PLS[2:0]=101 (falling edge)	2.47	2.58	2.69	
		PLS[2:0]=110 (rising edge)	2.66	2.78	2.9	
		PLS[2:0]=110 (falling edge)	2.56	2.68	2.8	
		PLS[2:0]=111 (rising edge)	2.76	2.88	3	
		PLS[2:0]=111 (falling edge)	2.66	2.78	2.9	
$V_{PVDhyst}^{(2)}$	PVD hysteresis	-	-	100	-	mV

1.4 低功耗

在系统或电源复位以后，微控制器处于运行状态。当 CPU 不需继续运行时，可以利用多种低功耗模式来节省功耗，例如等待某个外部事件时。

用户需要根据最低电源消耗、最快速启动时间和可用的唤醒源等条件，选定一个最佳的低功耗模式。

STM32F10xxx 有三种低功耗模式：

模式	进入	唤醒	对1.8V区域时钟的影响	对VDD区域时钟的影响	电压调节器
睡眠 (SLEEP-NOW或 SLEEP-ON-EXIT)	WFI	任一中断	CPU时钟关，对其他时钟和ADC时钟无影响	无	开
	WFE	唤醒事件			
停机	PDDS和LPDS位 +SLEEPDEEP位 +WFI或WFE	任一外部中断(在外部中断寄存器中设置)	关闭所有1.8V区域的时钟	HSI 和HSE的振荡器关闭	开启或处于低功耗模式(依据电源控制寄存器(PWR_CR)的设置)
待机	PDDS位 +SLEEPDEEP位 +WFI或WFE	WKUP引脚的上升沿、RTC闹钟事件、NRST引脚上的外部复位、IWDG复位			关

1.4.1 睡眠模式

SLEEP-NOW模式	说明
进入	在以下条件下执行WFI(等待中断)或WFE(等待事件)指令： - SLEEPDEEP = 0 和 - SLEEPONEXIT = 0 参考Cortex-M3系统控制寄存器。
退出	如果执行WFI进入睡眠模式： 中断：参考中断向量表(表54) 如果执行WFE进入睡眠模式： 唤醒事件：参考唤醒事件管理(第9.2.3节)
唤醒延时	无

SLEEP-ON_EXIT模式	说明
进入	在以下条件下执行WFI指令： - SLEEPDEEP = 0和 - SLEEPONEXIT = 1 参考Cortex™-M3系统控制寄存器
退出	中断：参考中断向量表(表54)
唤醒延时	无

1) 进入睡眠模式

通过执行 WFI (Wait for Interrupt) 或 WFE (Wait for Event) 指令进入睡眠状态。根据 Cortex™-M3 系统控制寄存器中的 **SLEEPONEXIT** 位的值, 有两种选项可用于选择睡眠模式进入机制:

(1) SLEEP-NOW: 如果 SLEEPONEXIT 位被清除, 当 WRI 或 WFE 被执行时, 微控制器立即进入睡眠模式。

(2) SLEEP-ON-EXIT: 如果 SLEEPONEXIT 位被置位, 系统从最低优先级的中断处理程序中退出时, 微控制器就立即进入睡眠模式。

注意: 在睡眠模式下, 所有的 **I/O 引脚** 都保持它们在运行时的状态。

2) 退出睡眠模式

如果执行 WFI 指令进入睡眠模式, 任意一个被嵌套向量中断控制器响应的外设中断都能将系统从睡眠模式唤醒。

如果执行 WFE 指令进入睡眠模式, 则一旦发生唤醒事件时, 微处理器都将从睡眠模式退出。

1.4.2 停机 (停止) 模式

停止模式是在 Cortex™-M3 的深睡眠模式基础上结合了外设的时钟控制机制, 在停止模式下电压调节器可运行在正常或低功耗模式。此时在 1.8V 供电区域的的所有时钟都被停止, PLL、HSI 和 HSE RC 振荡器的功能被禁止, SRAM 和寄存器内容被保留下来。

注意: 在停止模式下, 所有的 I/O 引脚都保持它们在运行模式时的状态。

1) 进入停止模式

在停止模式下, 通过设置电源控制寄存器(PWR_CR)的 LPDS 位使内部调节器进入低功耗模式, 能够降低更多的功耗。

(1) 如果正在进行闪存编程, 直到对内存访问完成, 系统才进入停止模式。

(2) 如果正在进行对 APB 的访问, 直到对 APB 访问完成, 系统才进入停止模式。

2) 退出停止模式

停止模式	说明
进入	在以下条件下执行WFI(等待中断)或WFE(等待事件)指令： – 设置Cortex-M3系统控制寄存器中的SLEEPDEEP位 – 清除电源控制寄存器(PWR_CR)中的PDDS位 – 通过设置PWR_CR中LPDS位选择电压调节器的模式 注：为了进入停止模式，所有的外部中断的请求位(挂起寄存器(EXTI_PR))和RTC的闹钟标志都必须被清除，否则停止模式的进入流程将会被跳过，程序继续运行。
退出	如果执行WFI进入停止模式： – 设置任一外部中断线为中断模式(在NVIC中必须使能相应的外部中断向量)。参见中断向量表(表54)。 如果执行WFE进入停止模式： – 设置任一外部中断线为事件模式。参见唤醒事件管理(第9.2.3节)。
唤醒延时	HSI RC唤醒时间 + 电压调节器从低功耗唤醒的时间。

1.4.3 待机模式

待机模式可实现系统的**最低功耗**。

该模式是在 Cortex-M3 深睡眠模式时关闭电压调节器。整个 1.8V 供电区域被断电。PLL、HSI 和 HSE 振荡器也被断电。SRAM 和寄存器内容丢失。只有**备份寄存器和待机电路**维持供电。这么说吧，能停的全停。

待机模式	说明
进入	在以下条件下执行WFI(等待中断)或WFE(等待事件)指令： – 设置Cortex™-M3系统控制寄存器中的SLEEPDEEP位 – 设置电源控制寄存器(PWR_CR)中的PDDS位 – 清除电源控制/状态寄存器(PWR_CSR)中的WUF位
退出	WKUP引脚的上升沿、RTC闹钟事件的上升沿、NRST引脚上外部复位、IWDG复位。
唤醒延时	复位阶段时电压调节器的启动。

注意：在待机模式下，所有的 I/O 引脚处于高阻态，除了以下的引脚：

- (1) 复位引脚（始终有效）
- (2) 当被设置为防侵入或校准输出时的 TAMPER 引脚
- (3) 被使能的唤醒引脚

1.5 低功耗案例 1：睡眠模式

1.5.1 需求描述

让 MCU 进入睡眠模式，然后通过串口发送消息来唤醒 MCU 退出睡眠模式。观察 LED 在进入睡眠模式后是否仍然开启。

1.5.2 软件设计（寄存器）



低功耗之睡眠模式
用到的寄存器详解.

1.5.2.1 main.c

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "Delay.h"

void enter_sleep_mode(void)
{
    /* 1. 设置普通睡眠模式 */
    SCB->SCR &= ~SCB_SCR_SLEEPDEEP;
    /* 2. 进入睡眠模式，等待中断唤醒 */
    __WFI();
}

int main(void)
{
    usart1_init();
    LED_Init();
    /* 开启 LED 灯 */
    LED_On();
    Delay_s(3);
    while (1)
    {
        printf("正常代码执行完毕，5s 后进入睡眠模式 \r\n");
        Delay_s(5);
        printf("开始睡眠 \r\n");
        enter_sleep_mode();
        Delay_s(1);
        printf("从睡眠模式被唤醒...\r\n");
    }
}
```

1.5.2.2 usart.h

```
#ifndef __USART_H
#define __USART_H

#include "stm32f10x.h"
#include "stdio.h"
/* 初始化串口 1 */
void usart1_init(void);
/*
    通过串口 1 发送数据:
*/
```

```

    参数 1: 要发送的数据
    参数 2: 要发送的数据的长度
*/
void send_string(uint8_t *str, uint32_t len);

/*
    从串口 1 接收一个字符
    返回值: 接收到的字符
*/
uint8_t receive_char(void);

/*
    向串口 1 发送一个字符
*/
void send_char(uint8_t c);

int fputc(int ch, FILE *f);
#endif

```

1.5.2.3 usart.c

```

#include "usart.h"

/* 初始化串口 1 */
void usart1_init(void)
{
    /* 1 开启时钟 1<<14 = 0x4000*/
    /* 1.1 开启 usart1 时钟 */
    RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
    /* 1.2 开启 GPIOA 时钟 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

    /* 2 配置引脚: PA9(Tx)和 PA10(Rx) */
    /* 2.1 PA9 配置为推挽输出 mode=11 cnf=10*/
    GPIOA->CRH |= (GPIO_CRH_MODE9 | GPIO_CRH_CNF9_1);
    /* 2.2 PA10 配置为浮空输入 */
    GPIOA->CRH |= GPIO_CRH_CNF10_0;

    /* 3 配置波特率为: 115200*/
    USART1->BRR = 0x271;

    /* 4 使能 usart1 的发送和接收 */
    USART1->CR1 |= (USART_CR1_TE | USART_CR1_RE);

    /* 5 使能 usart1 接收中断 */
}

```

```

USART1->CR1 |= USART_CR1_RXNEIE;

/* 6. 设置 NVIC */
NVIC_SetPriorityGrouping(4);
NVIC_SetPriority(USART1_IRQn, 2);
NVIC_EnableIRQ(USART1_IRQn);

/* 7 使能 usart1 */
USART1->CR1 |= USART_CR1_UE;
}
/*
    通过串口 1 发送数据:
    参数 1: 要发送的数据
    参数 2: 要发送的数据的长度

*/
void send_string(uint8_t *str, uint32_t len)
{
    uint32_t i;
    for (i = 0; i < len; i++)
    {
        send_char(str[i]);
    }
}

/*
    从串口 1 接收一个字符
    返回值: 接收到的字符
*/
uint8_t receive_char(void)
{
    /* 等待接收缓冲区 非空 */
    while ((USART1->SR & USART_SR_RXNE) == 0)
    {
    }
    return USART1->DR;
}

/*
    向串口 1 发送一个字符
*/
void send_char(uint8_t c)
{
    /* 等待发送缓冲区为空。 SR_TXE 为 1 表示已经移到移位寄存器， 0 表示还没有 */

```

```

while ((USART1->SR & USART_SR_TXE) == 0)
{
}
/* 把要发送的数据写入到数据寄存器 */
USART1->DR = c;
}
/* 中断服务函数 */
void USART1_IRQHandler(void)
{
    if (USART1->SR & USART_SR_RXNE)
    {
        /* 获取接收到的数据： 只取低 8 位 */
        uint8_t c = USART1->DR & 0xff;
        /* 回显给上位机 */
        send_char(c);
    }
    /* 可以不用清除中断标志位， 因为读取 DR 寄存器值的时候会自动清零 */
    // USART1->SR &= ~USART_SR_RXNE;
}

/*
printf 函数底层会调用 fputc
重定向 c 库函数 printf 到串口，重定向后可使用 printf 函数打印数据
*/
int fputc(int ch, FILE *f)
{
    send_char((uint8_t)ch);
    return (ch);
}

```

1.5.2.4 LED.h

```

#ifndef __LED_H
#define __LED_H
#include "stm32f10x.h"

/* 对 led 进行初始化 */
void LED_Init(void);

/* 点亮 led */
void LED_On(void);

/* 熄灭 led */
void LED_Off(void);

/* 翻转 led 的状态 */

```

```
void LED_Toggle(void);
```

```
#endif
```

1.5.2.5 LED.c

```
#include "LED.h"
```

```
/* 对 led 进行初始化 */
```

```
void LED_Init(void)
```

```
{
```

```
    /* 1. 使能 GPIOA 时钟 */
```

```
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN; // 1<<2
```

```
    /* 2. 设置 PA0 为通用推挽输出，速度 50MHz MODE=11 CNF=00 */
```

```
    GPIOA->CRL |= GPIO_CRL_MODE1;
```

```
    GPIOA->CRL &= ~GPIO_CRL_CNF1;
```

```
}
```

```
/* 点亮 led */
```

```
void LED_On(void)
```

```
{
```

```
    GPIOA->ODR &= ~GPIO_ODR_ODR1;
```

```
}
```

```
/* 熄灭 led */
```

```
void LED_Off(void)
```

```
{
```

```
    GPIOA->ODR |= GPIO_ODR_ODR1;
```

```
}
```

```
/* 翻转 led 的状态 */
```

```
void LED_Toggle(void)
```

```
{
```

```
    if (GPIOA->IDR & GPIO_IDR_IDR1)
```

```
    {
```

```
        LED_On();
```

```
    }
```

```
    else
```

```
    {
```

```
        LED_Off();
```

```
    }
```

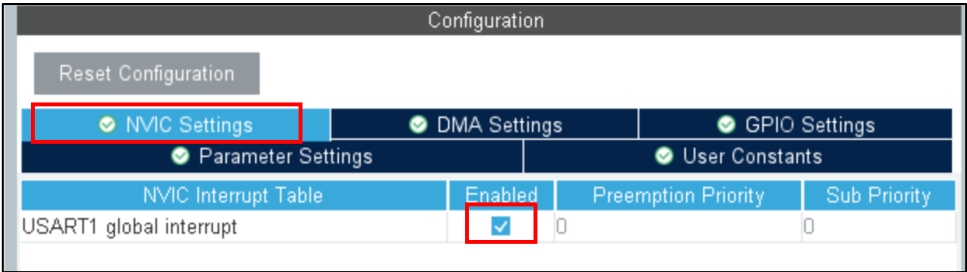
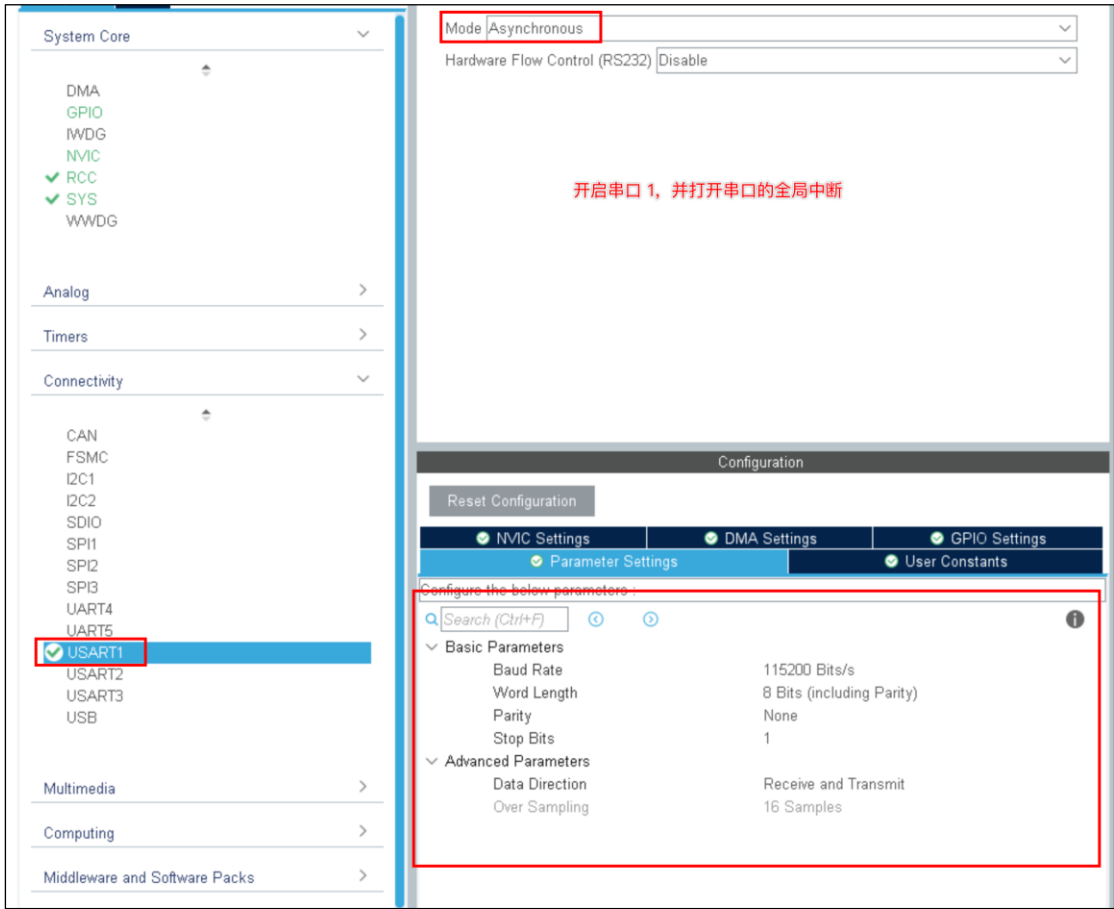
```
}
```

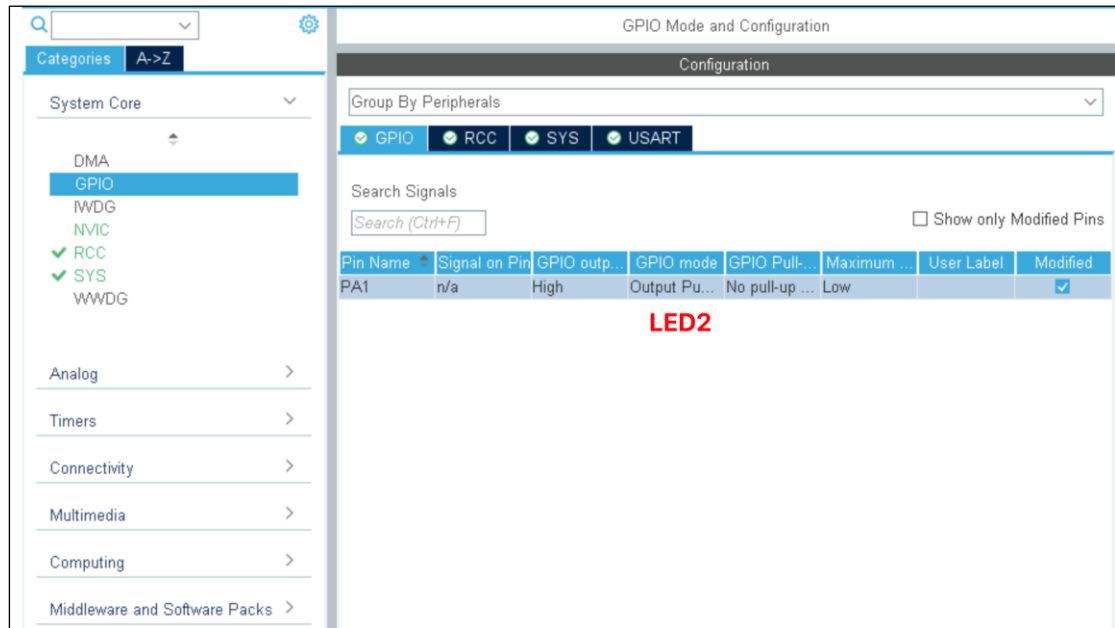
1.5.2.6 测试

```
正常代码执行完毕，5s后进入睡眠模式
开始睡眠
abc
从睡眠模式被唤醒...
正常代码执行完毕，5s后进入睡眠模式
开始睡眠
```

1.5.3 软件设计（HAL 库）

1.5.3.1 STM32CubeMX 设置





1.5.3.2 添加其他代码

```
uint8_t buff[100];

// 串口收到数据的时候回调这个函数
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        printf("%c", buff[0]);
        HAL_UART_Receive_IT(&huart1, buff, 1); // 继续接收
    }
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART1_UART_Init();
    /* USER CODE BEGIN 2 */
    printf("尚硅谷低功耗实验：睡眠模式\r\n");
    /* 点亮 LED */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET);
    // 利用串口中断接收消息
    HAL_UART_Receive_IT(&huart1, buff, 1);
    while (1)
    {
        HAL_Delay(3000);
    }
}
```

```

printf("正常代码执行完毕, 5s 后进入睡眠模式 \r\n");
HAL_Delay(5000);
printf("开始睡眠 \r\n");
/* 1. 暂停滴答定时器, 防止被自动唤醒 */
HAL_SuspendTick();
/* 2. 进入睡眠模式: 参数 1: 要不要关闭电压调节器(睡眠模式无效) 参数 2: 进入指令
wfi or wfe */
HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
/* 3. 被唤醒后要恢复滴答时钟 */
HAL_ResumeTick();
HAL_Delay(1);
printf("从睡眠模式被唤醒...\r\n");
}
}

```

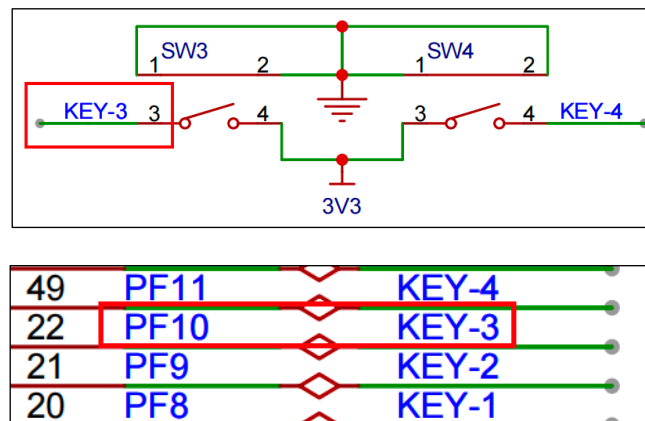
1.6 低功耗案例 2：停机模式

1.6.1 需求描述

让 MCU 进入停机模式。停机模式需要外部中断才能唤醒，所以要开启外部中断。（串口不是外部中断）

1.6.2 硬件电路设计

我们按下按键来产生外部中断。



1.6.3 软件设计（寄存器）



低功耗之停机模式
用到的寄存器详解.

复制实验 1 项目，新增外部中断相关配置。使用寄存器实现。

1.6.3.1 main.c

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "Delay.h"
#include "key.h"
#include "LED.h"

void enter_stop_mode(void)
{
    /* 1. 开启 PWR 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_PWREN;

    /* 2. 清除电源控制器中的 PDDS 位 进入深睡眠时，进入停机模式*/
    PWR->CR &= ~PWR_CR_PDDS;

    /* 3. 设置电压调节器的模式 0: 电压调节器开启 1: 电压调节器处于低功耗模式*/
    PWR->CR |= PWR_CR_LPDS;

    /* 4. 系统控制寄存器的深度睡眠模式*/
    SCB->SCR |= SCB_SCR_SLEEPDEEP;

    /* 5. 进入睡眠模式，等待被中断唤醒 */
    __WFI();
}

/**
 * @description: 从 stop 模式恢复后，重新设置系统时钟
 * @return {*}
 */
void config_system_clock(void)
{
    /* 1. 使能 HSE */
    RCC->CR |= RCC_CR_HSEON;

    /* 2. 由于时钟启用需要一定时间，需要等待 HSE 准备就绪*/
    while (!(RCC->CR & RCC_CR_HSERDY))
        ;

    /* 3. 使能 PLL */
    RCC->CR |= RCC_CR_PLLON;

    /* 4. 等待 PLL 准备就绪 */
    while (!(RCC->CR & RCC_CR_PLLRDY))
        ;
}
```

```

/*
    5. 选择 PLL 作为系统时钟源 CFGR_SW 位控制
        00: HSI 作为系统时钟;
        01: HSE 作为系统时钟;
        10: PLL 输出作为系统时钟;
        11: 不可用。
*/
RCC->CFGR |= RCC_CFGR_SW_1;
RCC->CFGR &= ~RCC_CFGR_SW_0;

/* 6. 等待 PLL 被选为系统时钟源 */
while (!(RCC->CFGR & RCC_CFGR_SWS_PLL))
    ;
}

/**
 * @description: 获取各个时钟频率
 */
void get_clock_freq(uint32_t *sys_clock,
                    uint32_t *AHB_clock,
                    uint32_t *APB1_clock,
                    uint32_t *APB2_clock)
{
    /* 1. 获取时钟源 RCC_CFGR_SWS 用来获取当前时钟源*/
    uint32_t system_clock_source = RCC->CFGR & RCC_CFGR_SWS;

    /* 2. 根据时钟源获取主时钟频率 */
    if (system_clock_source == RCC_CFGR_SWS_HSE)
    {
        *sys_clock = HSE_VALUE;
    }
    else if (system_clock_source == RCC_CFGR_SWS_HSI)
    {
        *sys_clock = HSI_VALUE;
    }
    else if (system_clock_source == RCC_CFGR_SWS_PLL)
    {
        /* 倍频系数 */
        uint32_t mull = ((RCC->CFGR & RCC_CFGR_PLLMULL) >> 18) + 2;
        *sys_clock = HSE_VALUE * mull; /* 默认情况 PLL 时用 HSE 的 9 倍频 */
    }

    /* 3. 获取 AHB 总线时钟频率 默认情况 AHB 等于系统频率*/

```

```

uint32_t hpre = (RCC->CFGR & RCC_CFGR_HPRE) >> 4;
if (hpre & 0x8) // 有分频
{
    *AHB_clock = *sys_clock >> ((hpre & 0x7) + 1);
}
else
{
    // 没有分频
    *AHB_clock = *sys_clock;
}

/* 4. APB1 的频率 默认是 系统频率 2 分频*/
uint32_t ppre1 = (RCC->CFGR & RCC_CFGR_PPRE1) >> 8;
if (ppre1 & 0x4) // 有分频
{
    *APB1_clock = *AHB_clock >> ((ppre1 & 0x3) + 1);
}
else
{
    // 没有分频
    *APB1_clock = *AHB_clock;
}

/* 4. APB2 的频率 默认是 系统频率*/
uint32_t ppre2 = (RCC->CFGR & RCC_CFGR_PPRE2) >> 11;
if (ppre2 & 0x4) // 有分频
{
    *APB2_clock = *AHB_clock >> ((ppre2 & 0x3) + 1);
}
else
{
    // 没有分频
    *APB2_clock = *AHB_clock;
}
}

int main(void)
{
    usart1_init();
    key_init();
    LED_Init();
    LED_On();
    Delay_s(1);
    while (1)

```

```

{
    printf("正常代码执行完毕，5s 后进入停机模式\r\n");
    Delay_s(5);
    printf("开始进入停机模式，按下按键唤醒\r\n");
    Delay_ms(1);
    enter_stop_mode();
    /* 获取唤醒后的时钟频率 */
    uint32_t pre_system_clock, pre_AHB_clock, pre_APB1_clock, pre_APB2_clock;
    get_clock_freq(&pre_system_clock, &pre_AHB_clock, &pre_APB1_clock,
&pre_APB2_clock);

    /*
        从停机模式恢复后，会启用 HSI（高速内部时钟 8M）作为系统时钟，与我们正常使用的
        有差别的。
        所以需要恢复到 HSE（高速外部时钟）
    */
    config_system_clock();

    /* 获取重置后的时钟频率 */
    uint32_t post_system_clock, post_AHB_clock, post_APB1_clock,
post_APB2_clock;
    get_clock_freq(&post_system_clock, &post_AHB_clock, &post_APB1_clock,
&post_APB2_clock);

    printf("从停机模式被唤醒...\r\n");

    printf("重置前的时钟频率: pre_system_clock=%d, pre_AHB_clock=%d,
pre_APB1_clock=%d, pre_APB2_clock=%d \r\n",
        pre_system_clock,
        pre_AHB_clock,
        pre_APB1_clock,
        pre_APB2_clock);

    printf("重置后的时钟频率: post_system_clock=%d, post_AHB_clock=%d,
post_APB1_clock=%d, post_APB2_clock=%d \r\n",
        post_system_clock,
        post_AHB_clock,
        post_APB1_clock,
        post_APB2_clock);
}
}

```

1.6.3.2 key.h

```

#ifndef __KEY_H
#define __KEY_H

```

```

/* 初始化按键： 外部中断*/
void key_init(void);

#endif

```

1.6.3.3 key.c

```

#include "key.h"
#include "stm32f10x.h"
#include "usart.h"

/* 初始化按键： 外部中断*/
void key_init(void)
{
    /* 1 开启时钟 (EXTI 和 NVIC 时钟始终开启, 无需手动开启)*/
    /* 1.1 开启 GPIOF 时钟 (PF10) */
    RCC->APB2ENR |= RCC_APB2ENR_IOPFEN;
    /* 1.2 开启 AFIO 时钟 */
    RCC->APB2ENR |= RCC_APB2ENR_AFIOEN;

    /* 2 配置 PF10 设置为输入 下拉(cnf: 10 mode: 00) 。另外还需要把 ODR 寄存器对应的位设
    为 0*/
    GPIOF->CRH &= ~GPIO_CRH_MODE10;
    GPIOF->CRH |= GPIO_CRH_CNF10_1;
    GPIOF->CRH &= ~GPIO_CRH_CNF10_0;
    GPIOF->ODR &= ~GPIO_ODR_ODR10;

    /* 3 配置 AFIO, 用于引脚复用为外部中断 PF10 */
    AFIO->EXTICR[2] |= AFIO_EXTICR3_EXTI10_PF;

    /* 4. 配置 EXTI */
    /* 4.1 PF10 的上升沿触发中断 */
    EXTI->RTSR |= EXTI_RTSR_TR10;
    /* 4.2 开启 EXTI10 线 */
    EXTI->IMR |= EXTI_IMR_MR10;

    /* 5. 配置 NVIC */
    /* 5.1 配置中断优先级组 */
    NVIC_SetPriorityGrouping(4);
    /* 5.2 配置中断优先级 */
    NVIC_SetPriority(EXTI15_10_IRQn, 2);
    /* 5.3 使能中断线 15-10 中断 */
    NVIC_EnableIRQ(EXTI15_10_IRQn);
}

```

```
/* 中断服务函数 */
void EXTI15_10_IRQHandler(void)
{
    /* 清除中断标志 */
    EXTI->PR |= EXTI_PR_PR10;
}
```

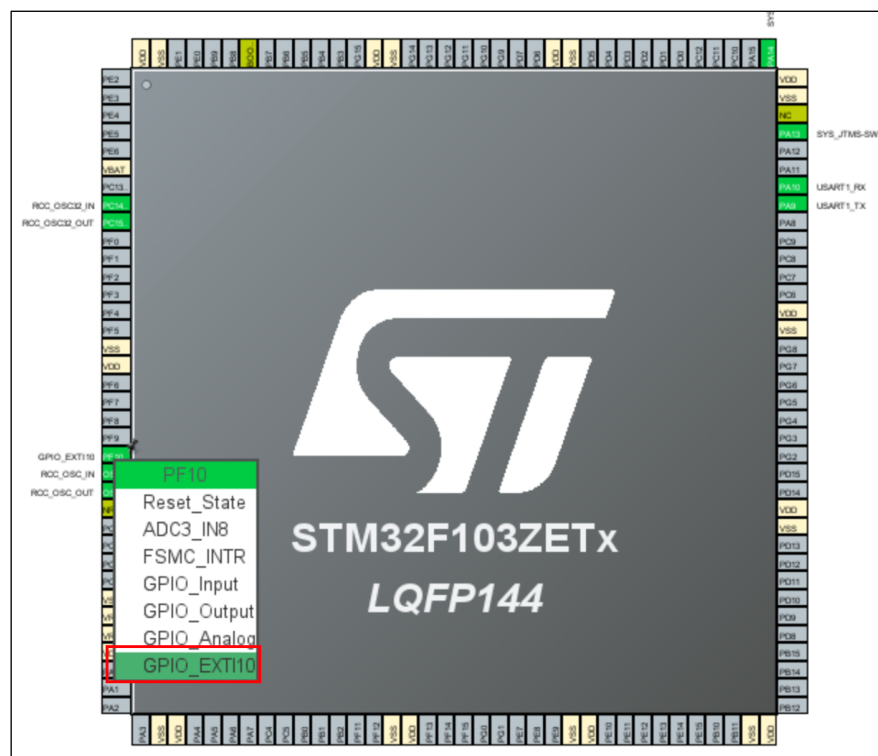
1.6.3.4 测试

正常代码执行完毕，开始进入待机模式，按下按键唤醒	5s后进入待机模式
从待机模式被唤醒，重新配置时钟频率，正常代码执行完毕，开始进入待机模式，按下按键唤醒	pre_system_clock=8000000, pre_AHB_clock=8000000, pre_APB1_clock=4000000, pre_APB2_clock=8000000 post_system_clock=72000000, post_AHB_clock=72000000, post_APB1_clock=36000000, post_APB2_clock=72000000
5s后进入待机模式，按下按键唤醒	

1.6.4 软件设计 (HAL 库)

复制实验 2，新增外部中断相关配置。

1.6.4.1 STM32CubeMX 的配置



Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin Na...	Signal on ...	GPIO outp...	GPIO mode	GPIO Pull...	Maximum ...	User Label	Modified
PA1	n/a	High	Output Pu...	No pull-up...	Low		☒
PF10	n/a	n/a	External I...	Pull-down	n/a		☒

PF10 Configuration :

外部中断上升沿检测

GPIO mode

External Interrupt Mode with Rising edge trigger detection

GPIO Pull-up/Pull-down

Pull-down

下拉

User Label

☒ GPIO

☒ RCC

☒ SYS

☒ USART

☒ NVIC

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
EXTI line[15:10] interrupts	☒	0	0

开启中断

1.6.4.2 添加其他代码

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART1_UART_Init();
    /* USER CODE BEGIN 2 */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET);
    printf("尚硅谷低功耗实验:停机模式 HAL 库\r\n");
    while (1)
    {
```

```

{
    printf("正常任务执行完毕,5s后进入 停止 模式... \r\n");
    HAL_Delay(5000);
    printf("开始进入 停止 模式... \r\n");
    // 1. 暂停滴答定时器（防止退出停止模式）
    HAL_SuspendTick();
    // 2. 进入停止模式：
    // 参数 1：要不要关电压调节器
    // 参数 2：进入指令：wfi 还是 wfe
    HAL_PWR_EnterSTOPMode(PWR_MAINREGULATOR_ON, PWR_STOPENTRY_WFI);

    // 3. 重新配置系统时钟(如果不配置时钟，串口打印不正常，因为时钟频率变化了，波特率计算也不对了)
    SystemClock_Config();

    // 4. 恢复滴答定时器
    HAL_ResumeTick();
    printf("系统被唤醒...\r\n");
}
}

```

1.7 低功耗案例 3：待机模式

1.7.1 需求描述

寄存器操作进入待机模式。待机模式的唤醒方式比较有限。我们这次使用 WKUP 引脚的**上升沿**唤醒。PA0 就是 WKUP 引脚。

当然 PA0 仍然需要工作在下拉输入模式，只有这样当按键按下时才会有一个上升沿。

WKUP引脚的上升沿、RTC闹钟事件、NRST引脚上的外部复位、IWDG复位

由于我们电路中 PA0 已经连接了 LED1，所以要产生上升沿需要先用杜邦线连接 GND，然后再连接 3.3V 电源即可。

另外也可以使用 NRST 引脚实现唤醒。

1.7.2 软件设计（寄存器）



低功耗之待机模式 用到的寄存器详解.

Copy 实验 3，进行修改。

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "Delay.h"
#include "led.h"

/**
 * @description: 进入待机模式
 */
void enter_standby_mode(void)
{
    /* 1. 使能 PA0 的唤醒功能 */
    PWR->CSR |= PWR_CSR_EWUP; /* 引脚会被强制设置下拉输入 */

    /* 3. CPU 进入深睡眠时进入待机模式。 */
    PWR->CR |= PWR_CR_PDDS;

    /* 4. 系统控制寄存器的深度睡眠模式*/
    SCB->SCR |= SCB_SCR_SLEEPDEEP;

    /* 5. 进入待机模式，等待唤醒 */
    __WFI();
}

int main(void)
{
    led_init();
    led_on();
    usart1_init();

    /* 1. 开启 PWR 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_PWREN;

    /* 检测复位来源：按下 reset 和从待机恢复都会重新执行 main 函数 */
    /* PWR_CSR_SBF 是待机标志位：1 表示曾经待机过 */
    if (PWR->CSR & PWR_CSR_SBF)
    {
        printf("待机唤醒复位\r\n");
        /* 1. 清除待机标志位,然后才可以进入待机，通过 CR_CSBF 写 1 清除*/
    }
}
```

```

PWR->CR |= PWR_CR_CSBF;
/* 2. 清除唤醒标记, 通过 CR_CWUF*/
PWR->CR |= PWR_CR_CWUF;
}
else
{
    printf("非待机唤醒复位 \r\n");
}

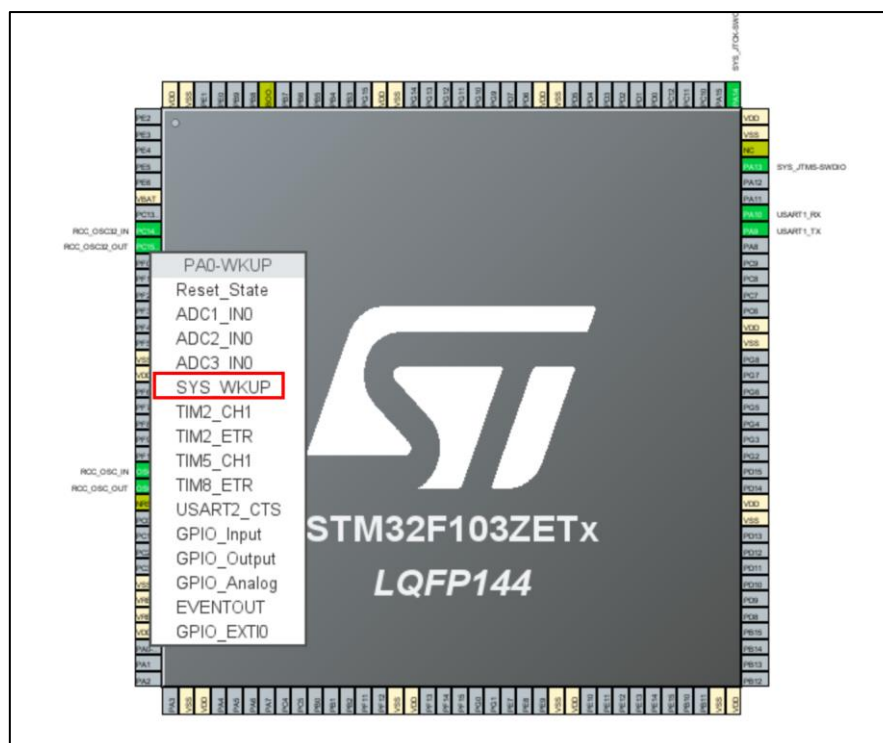
while (1)
{
    printf("正常代码执行完毕, 3s 后进入待机模式\r\n");
    Delay_s(3);
    printf("开始进入待机模式, 所有灯光将关闭, 按下按键唤醒\r\n");
    Delay_ms(1);
    enter_standby_mode();
}
}

```

注意: 待机期间是不能下载程序的, 需要唤醒之后再烧录。所以务必注意一定要加延时, 在这期间烧录程序, 否则一启动就进入待机模式, 就没有办法烧录程序了。

1.7.3 软件设计 (HAL 库)

1.7.3.1 STM32CubeMX 配置



1.7.3.2 添加其他代码

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART1_UART_Init();

    // 检测复位来源。 当从待机恢复的时候，会重新开始执行 main 函数
    if (__HAL_PWR_GET_FLAG(PWR_FLAG_SB) == SET) // 从待机恢复
    {
        // 清除待机标志位
        __HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB);
        printf("\r\n 待机唤醒复位 \r\n");
    }
    else
    {
        printf("\r\n 非待机唤醒复位 \r\n");
    }
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET);
    while (1)
    {
        printf("正常代码执行完毕，5s 后进入待机模式... \r\n");
        HAL_Delay(5000);
        // 1. 清除唤醒标记位
        __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
        // 2. 使能 PA0 的待机唤醒功能
        HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1);
        // 3. 停止滴答定时器
        HAL_SuspendTick();
        // 4. 进入待机模式
        HAL_PWR_EnterSTANDBYMode();
    }
}
```

第 2 章 RTC（实时时钟）

2.1 RTC 简介

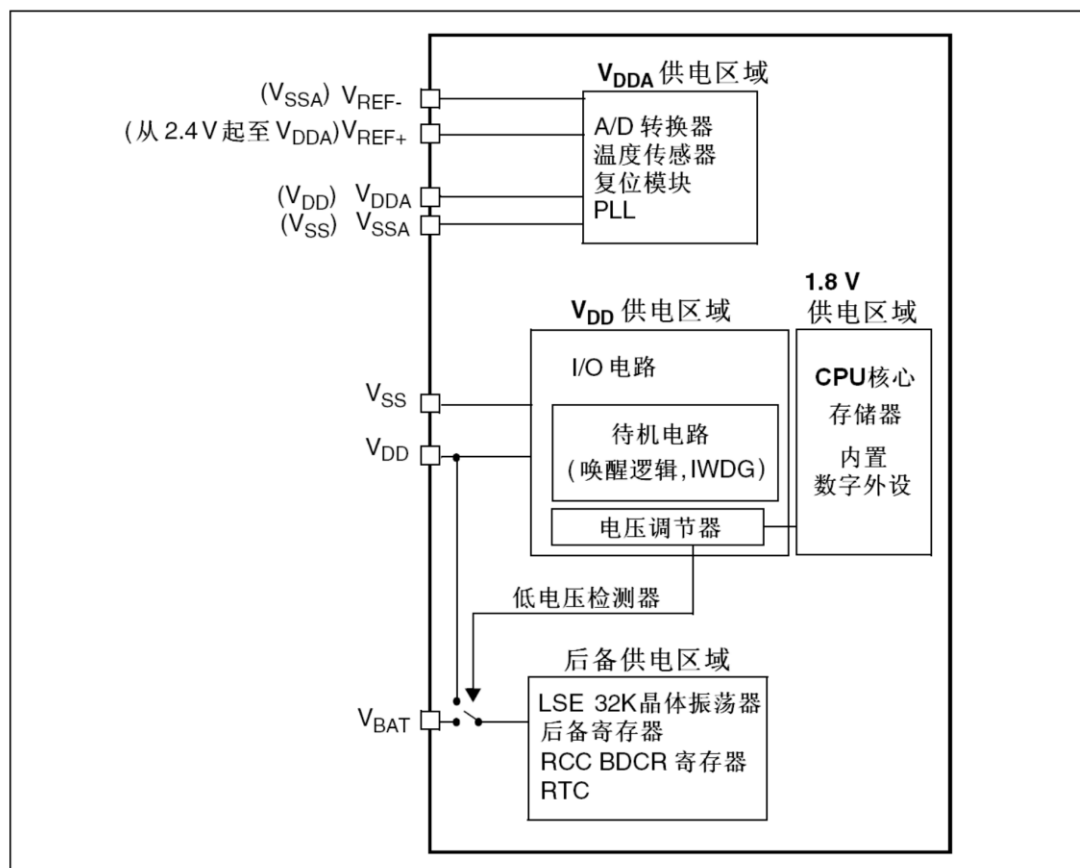
RTC（Real Time Clock，实时时钟）。是一个掉电后仍然可以继续运行的独立定时器。

RTC 模块拥有一个连续计数的计数器，在相应的软件配置下，可以提供时钟日历的功能。修改计数器的值可以重新设置当前时间和日期 RTC 还包含用于管理低功耗模式的自动唤醒单元。

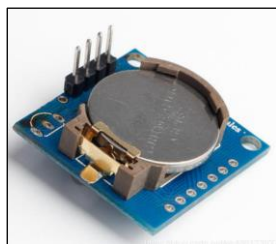
RTC 实质是一个掉电后还继续运行的定时器，从定时器的角度来看，相对于通用定时器 TIM 外设，它的功能十分简单，只有计时功能（也可以触发中断）。但其高级指出也就在于掉电之后还可以正常运行。

掉电能运行，不是说它不需要电能，而是它有一套专门的供电系统。

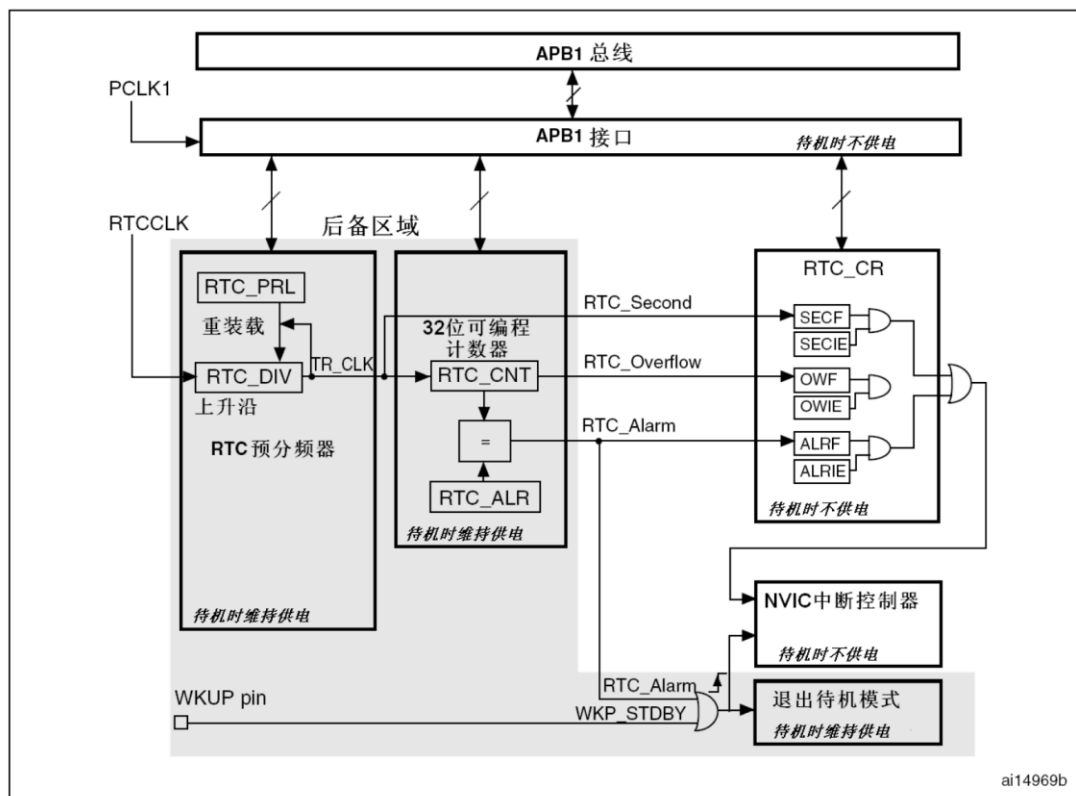
2.2 掉电继续运行



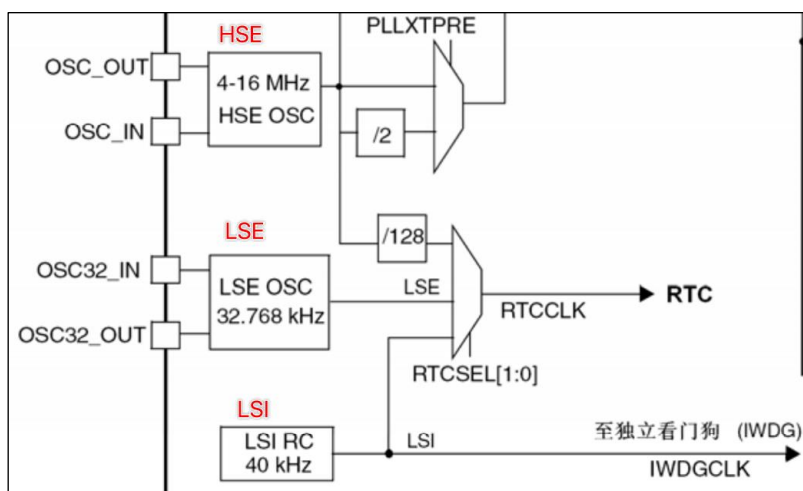
这里所说的掉电是指的 V_{DD} 掉电。 V_{DD} 掉电后，由 V_{BAT} 负责给后备区域供电。 V_{BAT} 由电路中的纽扣电池来供电，一般的电路设计中都会加上备用电池。只要这个电池有电，则后备区域就一直会有电供应。



2.3 RTC 功能框图



1) RTC 的时钟源



RTC 有 3 路时钟来源：HSE (8MHz) /128, LSE (32.768KHz), LSI (40KHz)。其中，如果 使用 HSE 或 LSI 的话，当主电源掉电的话，这两个始终都会受到影响，RTC 就无法正

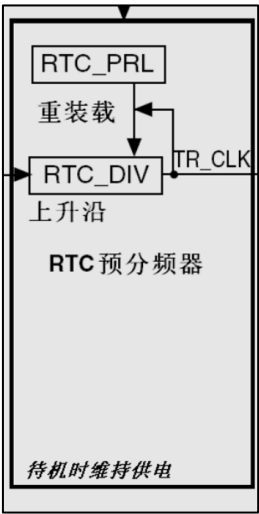
常工作。所以，一般的通用做法是使用 **LSE**。2 个原因：一是 LSE 不受主电源掉电的影响，二是它的频率是我们都是选择 32768Hz，正好是 2^{15} ，分频容易实现。

2) APB1 接口

用来和 APB1 总线相连。此单元还包含一组 16 位寄存器，可通过 APB1 总线对其进行读写操作。APB1 接口由 APB1 总线时钟驱动，用来与 APB1 总线连接。

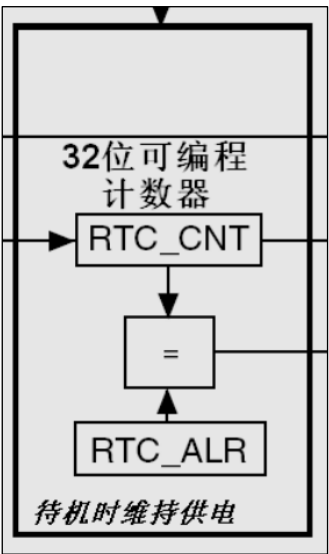
通过 APB1 接口可以访问 RTC 的相关寄存器（预分频值，计数器值，闹钟值）。

3) RTC 预分频模块



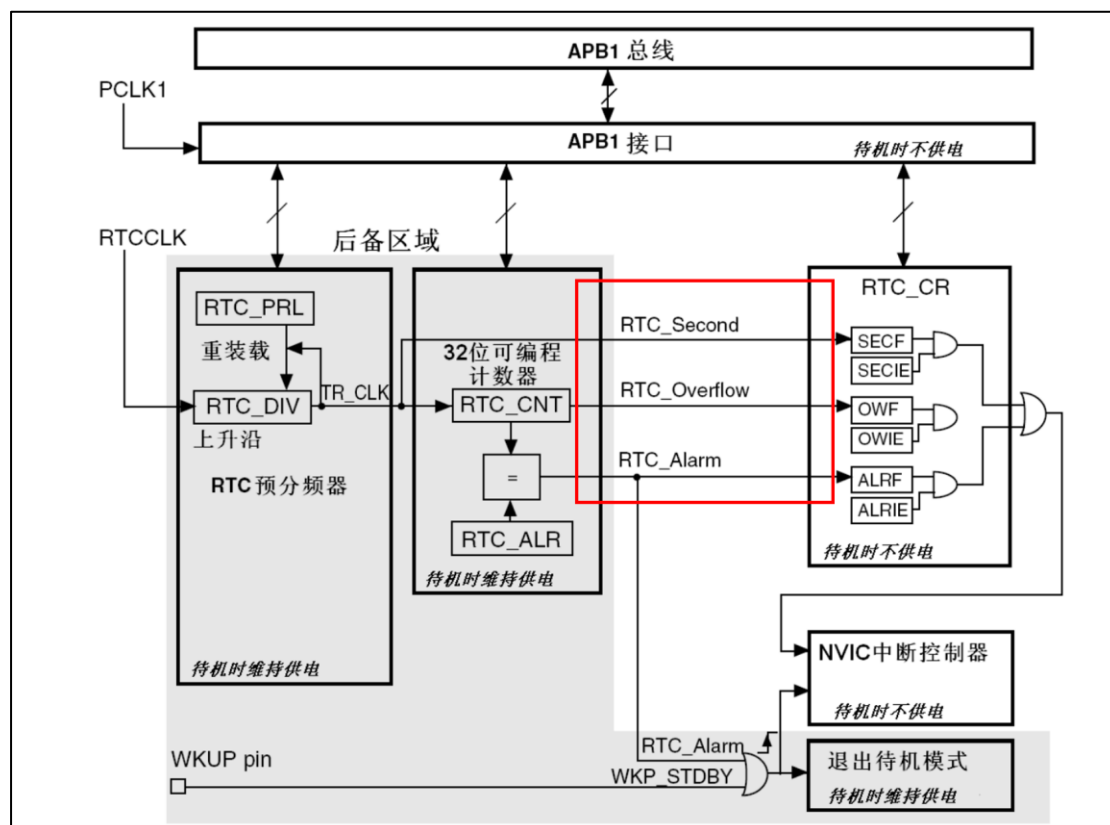
这个模块是 RTC 预分频模块，属于后备区域，V_{DD} 掉电后，可以在 V_{BAT} 下继续运行。包含了一个 20 位的可编程分频器（RTC 预分频器）。它可编程产生 1 秒的 RTC 时间基准 TR_CLK。如果在 RTC_CR 寄存器中设置了相应的允许位，则在每个 TR_CLK 周期中 RTC 产生一个中断（秒中断）。

4) 32 位可编程计数器



这个模块也属于后备区域，是一个 32 位的可编程计数器，可被初始化为当前的系统时间。一个 32 位的时钟计数器，按秒钟计算，可以记录 4294967296 秒，约合 136 年左右，作为一般应用，这已经是足够的了。

5) 中断



从图中可以看到一共有 3 个中断：

- (1) 秒中断：每计时 1s 产生一次中断。
- (2) 计数器溢出中断。136 年才会产生溢出，一般用不上。
- (3) RTC 闹钟中断。RTC_CNT 和 RTC_ALR 会比较相等，如果相等表示闹钟时间到，会产生闹钟中断。

第 3 章 备份寄存器（BKP）

3.1 备份寄存器介绍

3.1.1 备份寄存器简介

BKP（backup register，备份寄存器）。备份寄存器是 42 个 16 位的寄存器，可用来存储 84 个字节的用户应用程序数据。他们处在备份域里，当 V_{DD} 电源被切断，他们仍然由 V_{BAT} 维持供电。

当系统在待机模式下被唤醒，或系统复位或电源复位时，他们也不会被复位。

此外，BKP 控制寄存器用来管理**侵入检测**和**RTC 校准**功能。

复位后，对备份寄存器和 RTC 的访问被禁止，并且备份域被保护以防止可能存在的意外的写操作。执行以下操作可以使能对备份寄存器和 RTC 的访问：

（1）通过设置寄存器 RCC_APB1ENR 的 PWREN 和 BKPEN 位来打开电源和后备接口的时钟

（2）电源控制寄存器(PWR_CR)的 DBP 位来使能对后备寄存器和 RTC 的访问

3.1.2 侵入检测功能

当 TAMPER（PC13）引脚上的信号从 0 变成 1 或者从 1 变成 0（取决于备份控制寄存器 BKP_CR 的 TPAL 位），会产生一个侵入检测事件。侵入检测事件将所有数据备份寄存器内容清除。

设置 BKP_CSR 寄存器的 TPIE 位为"1"，当检测到侵入事件时就会产生一个中断。

3.1.3 RTC 校准

为方便测量，RTC 时钟可以经 64 分频输出到侵入检测引脚 TAMPER 上。通过设置 RTC 校验寄存器（BKP_RTCCR）的 CCO 位来开启这一功能。

3.1.4 备份数据寄存器

用来存储**用户数据**。一共 42 个 16 位寄存器（大容量和互联型是 42 个，中小容量产品是 20 个），可以存 84 个字节的数据。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
D[15:0]															
R'W R'W R'W R'W R'W R'W R'W R'W R'W R'W R'W R'W R'W R'W R'W R'W															
位15:0		D[15:0]: 备份数据 这些位可以被用来写入用户数据。 注意: BKP_DRx寄存器不会被系统复位、电源复位、从待机模式唤醒所复位。 它们可以由备份域复位来复位或(如果侵入检测引脚TAMPER功能被开启时)由侵入引脚事件复位。													

3.1.5 其他寄存器

RTC 时钟校准寄存器，备份控制寄存器，备份控制/状态寄存器。参考手册 49 页。

3.2 备份寄存器案例：读写备份寄存器

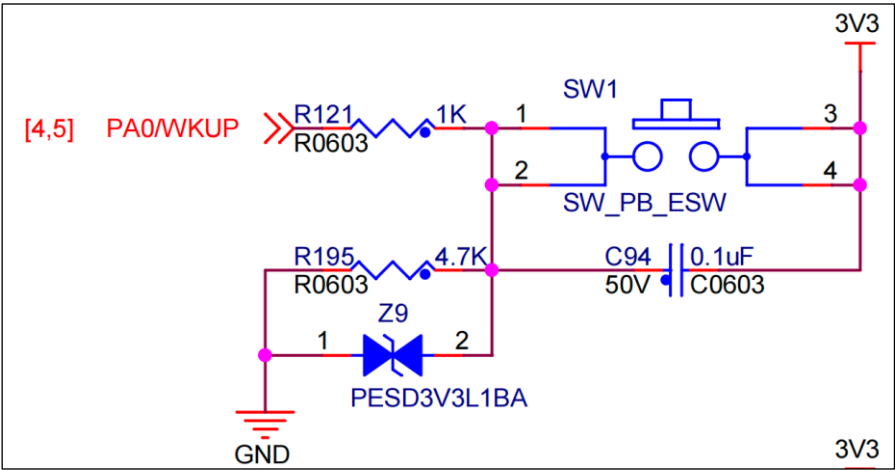
3.2.1 需求描述

基于寄存器操作。程序启动后向后备寄存器写入值，按下按键 1 从备份寄存器读出数据，验证是否一致。

Reset 设备，然后按下按键 1 是否能读到数据，验证掉电不丢失。

关机，再启动，然后按下按键 1 是否能读到数据，验证掉电不丢失。

3.2.2 硬件电路设计



3.2.3 软件设计（寄存器）



备份寄存器案例用
到的寄存器详解.doc

3.2.3.1 main.c

```
#include "stm32f10x.h" // Device header
```

```

#include "usart.h"
#include "Delay.h"
#include "key.h"
#include "bkp.h"
int main(void)
{
    usart1_init();
    printf("尚硅谷后备寄存器实验 基于寄存器实现...\r\n");
    key_init();

    /* 初始化 RTC 和后备寄存器 */
    rtc_bkp_init();

    /* 向后备寄存器写入值，只写一次，注释掉。 然后测试 rest 和掉电重启数据是否丢失 */
    //BKP->DR1 = 9999;

    while (1)
    {
    }
}
/* 按键按下时的中断服务函数 */
void EXTI15_10_IRQHandler(void)
{
    printf("DK1 = %d \r\n", BKP->DR1);
    /* 清除中断标志 */
    EXTI->PR |= EXTI_PR_PR10;
}

```

3.2.3.2 bkp.h

```

#ifndef __BKP_H
#define __BKP_H

#include "stm32f10x.h"
void rtc_bkp_init(void);

#endif

```

3.2.3.3 bkp.c

```

#include "bkp.h"

/**
 * @description: 初始化 rtc 和后备寄存器
 * @return {*}
 */
void rtc_bkp_init(void)

```

```

{
    /* 1. 使能 PWR 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_PWREN;

    /*
        2. 允许先 RTC 寄存器和后备寄存器写入数据 PWR_CR_DBP 位
            0: 禁止写入 RTC 和后备寄存器
            1: 允许写入 RTC 和后备寄存器
    */
    PWR->CR |= PWR_CR_DBP;

    /* 3. 使能 BKP 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_BKPEN;

    /* 4. 如果已经向 DR1 寄存器写入过值，证明备份域已经复位过。可以直接返回了 */
    if(BKP->DR1) return;

    /* 5. 复位备份域 */
    RCC->BDCR |= RCC_BDCR_BDRST;

    /* 6. 解除备份域复位 */
    RCC->BDCR &= ~RCC_BDCR_BDRST;

    /* 7. 使能 RTC 时钟 */
    RCC->BDCR |= RCC_BDCR_RTCEN;

    /* 8. 使能 LSE */
    RCC->BDCR |= RCC_BDCR_LSEON;

    /*
        9. 使用 LSE(外部低俗时钟) 作为 RTC 时钟
            00: 无时钟;
            01: LSE 振荡器作为 RTC 时钟;
            10: LSI 振荡器作为 RTC 时钟;
            11: HSE 振荡器在 128 分频后作为 RTC 时钟。

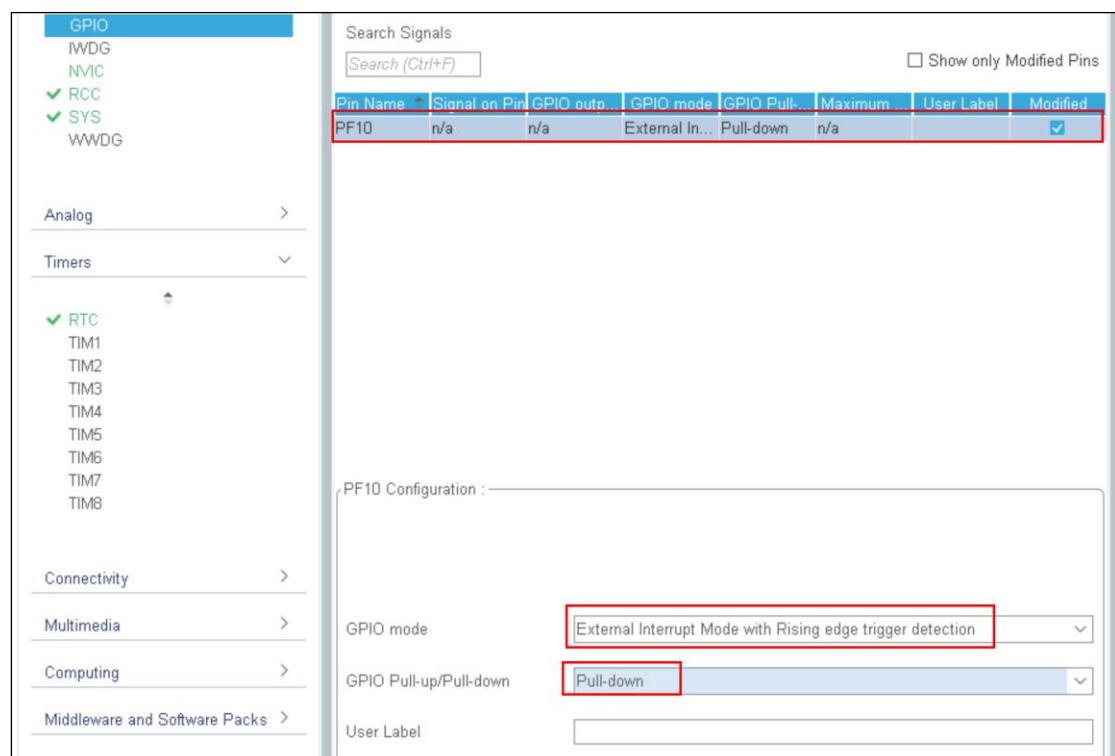
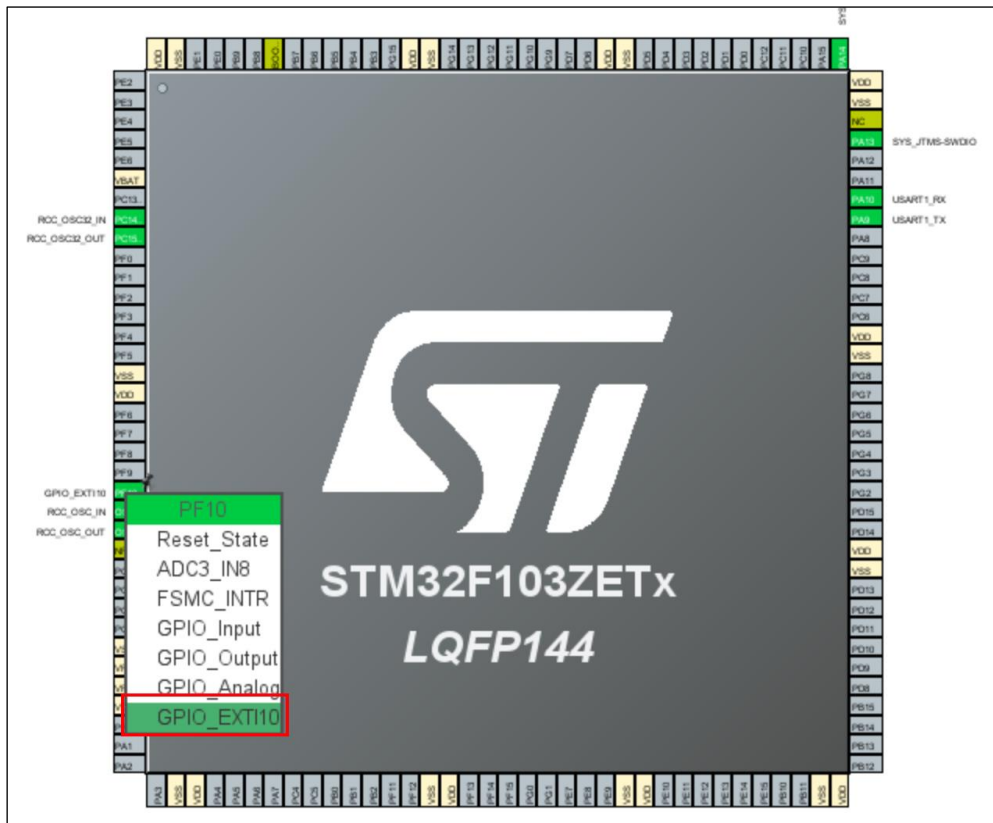
        如果要修改时钟源:
            1. 先复位后备域(所有数据丢失)复位后变为 00
            2. 这两位必须同时更改。如果先改高位，则低位就无法更改了。

        一旦 RTC 时钟选定，在备份域复位之前就不能再更改了。
    */
    RCC->BDCR |= (0x1 << 8);
}

```

3.2.4 软件设计（HAL 库）

3.2.4.1 STM32CubeMX 配置



☒ GPIO	☒ RCC	☒ SYS	☒ USART	☒ NVIC		
NVIC Interrupt Table				Enabled	Preemption Priority	Sub Priority
EXTI line[15:10] interrupts				☒	0	0

Categories

A-Z

System Core

DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG

Analog

Timers

RCC

TIM1
TIM2
TIM3
TIM4
TIM5
TIM6
TIM7
TIM8

RTC Mode and Configuration

Mode

☒ Activate Clock Source 激活 RTC 时钟源
☐ Activate Calendar
RTC OUT Disable
Tamper

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

Configure the below parameters:

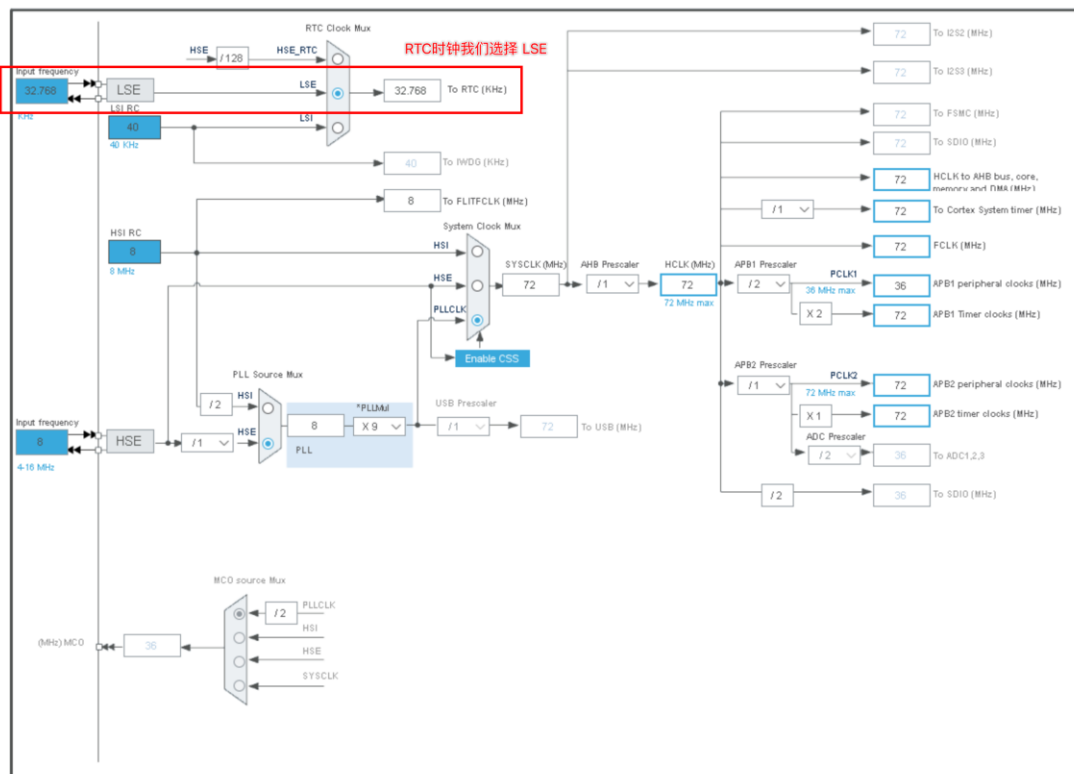
Calendar Time

Data Format

BCD data format

General

Auto Predivider Calculation Enabled
Asynchronous Predivider value Automatic Predivider Calculation Enabled
Output Alarm pulse signal on the TAMPER pin



3.2.4.2 添加其他代码

```
// 按键按下时的中断处理函数
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_10)
```

```
{
    uint32_t data = HAL_RTCEx_BKUPRead(&hrtc, RTC_BKP_DR1);
    printf("从后备寄存器读取的值是: %d \r\n", data);
}
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_RTC_Init();
    MX_USART1_UART_Init();
    // 向后备寄存器写入值,为了测试,只写一次,第一次写入之后,注释掉此代码
    // HAL_RTCEx_BKUPWrite(&hrtc, RTC_BKP_DR1, 1314);

    while (1)
    {
    }
}
```

第 4 章 RTC 实验

4.1 RTC 案例 1：使用闹钟唤醒待机模式

4.1.1 需求描述

执行完毕正常代码之后，让 MCU 进入待机模式，设置闹钟，自动让 MCU 从待机模式中被唤醒。可以用 led 点亮熄灭显示是否唤醒。

应用场景：比如设计一个野外温度自动采集的设备，规定每小时采集一次温度，就可以定义一个 1 小时的闹钟，定时唤醒，采集温度，采集完进入待机模式，可以大大降低设备功耗。

4.1.2 软件设计（寄存器）



RTC案例之闹钟唤醒待机模式用到的寄存器

4.1.2.1 main.c

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "Delay.h"
#include "led.h"
#include "alarm.h"

/**
 * @description: 进入待机模式
 */
void enter_standby_mode(void)
{
    /* 1. 使能 PA0 的唤醒功能 */
    // PWR->CSR |= PWR_CSR_EWUP; /* 引脚会被强制设置下拉输入 */

    /* 3. CPU 进入深睡眠时进入待机模式。 */
    PWR->CR |= PWR_CR_PDDS;

    /* 4. 系统控制寄存器的深度睡眠模式*/
    SCB->SCR |= SCB_SCR_SLEEPDEEP;

    /* 5. 进入待机模式，等待唤醒 */
    __WFI();
}
```

```

}

int main(void)
{

    LED_Init();
    LED_On();
    usart1_init();
    Alarm_Init();

    /* 1. 开启 PWR 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_PWREN;

    /* 检测复位来源：按下 reset 和从待机恢复都会重新执行 main 函数 */
    /* PWR_CSR_SBF 是待机标志位：1 表示曾经待机过 */
    if (PWR->CSR & PWR_CSR_SBF)
    {
        /* 1. 清除待机标志位,然后才可以进入待机，通过 CR_CSBF 写 1 清除*/
        PWR->CR |= PWR_CR_CSBF;
        /* 2. 清除唤醒标记，通过 CR_CWUF*/
        PWR->CR |= PWR_CR_CWUF;
    }

    while (1)
    {
        printf("温度测试完毕,5s 后进入待机....\r\n");
        Delay_s(5);
        printf("开始进入待机模式,所有灯光将关闭...\r\n");
        /* 设置 20s 后执行的闹钟 */
        Alarm_SetXSecond(20);
        Delay_ms(1);
        enter_standby_mode();
    }
}

```

4.1.2.2 alarm.h

```

#ifndef __ALARM_H
#define __ALARM_H

#include "stm32f10x.h"

void Alarm_Init(void);
void Alarm_SetXSecond(uint32_t x);
#endif

```

4.1.2.3 alarm.c

```
#include "alarm.h"

/**
 * @description: 闹钟的初始化
 * @return {*}
 */
void Alarm_Init(void)
{

    /* 1. 使能 PWR 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_PWREN;

    /* 2. 使能 BKP 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_BKPEN;

    /*
        3. 允许向 RTC 寄存器和后备寄存器写入数据 PWR_CR_DBP 位
            0: 禁止写入 RTC 和后备寄存器
            1: 允许写入 RTC 和后备寄存器
    */
    PWR->CR |= PWR_CR_DBP;

    /* 4. 使能 RTC 时钟 */
    RCC->BDCR |= RCC_BDCR_RTCEN;

    /* 5. 使能 LSE 外部低速时钟 */
    RCC->BDCR |= RCC_BDCR_LSEON;

    /* 6. 等待 RTC 进入初始化状态(上次的写操作已经完成) */
    while (!(RTC->CRL & RTC_CRL_RTOFF))
        ;

    /* 7. 进入配置模式. 允许配置 RTC 的计数器, 预分频系数和闹钟寄存器。 ----- */
    RTC->CRL |= RTC_CRL_CNF;

    /*
        8. 设置 RTC 预分频系数: 我们产生一个 1s 的信号, 让计数器每 s 加 1
            时钟频率是 32768Hz, 所以预分频系数应该是 32768-1=0x7FFF
    */
    RTC->PRLH = 0x0;
    RTC->PRLL = 0x7FFF;

    /* 9. 退出配置模式。前面的数据才会真正更新到寄存器中 -----*/
```

```

    RTC->CRL &= ~RTC_CRL_CNF;
}

/**
 * @description: 设置一个 x 秒之后到时的闹钟
 */
void Alarm_SetXSecond(uint32_t x)
{
    /* 1. 先清除闹钟标志 */
    RTC->CRL &= ~RTC_CRL_ALRF;

    /* 2. 等待 RTC 进入初始化状态 */
    while (!(RTC->CRL & RTC_CRL_RTOFF))
        ;

    /* 3. 进入配置模式 */
    RTC->CRL |= RTC_CRL_CNF;

    /* 4. 等待秒标志位 */
    while (!(RTC->CRL & RTC_CRL_SECF))
        ;

    /* 5. RTC 计数器寄存器的值。我们可以从 1 开始计数，则大概 136 年才会溢出（32 位计数器）
    */
    RTC->CNTH = 0;
    RTC->CNTL = 0;

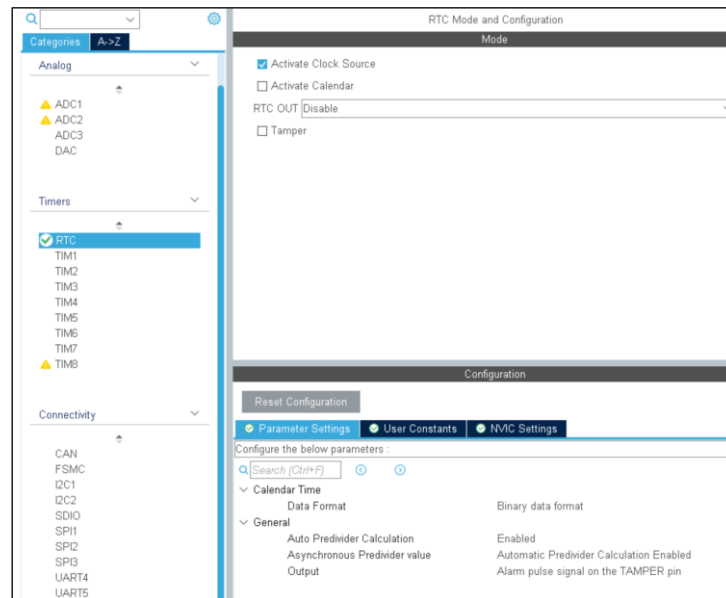
    /* 6. 设置闹钟： */
    x -= 1;
    RTC->ALRH = (x >> 16) & 0xFFFF;
    RTC->ALRL = x & 0xFFFF;

    /* 7. 退出配置模式 */
    RTC->CRL &= ~RTC_CRL_CNF;
}

```

4.1.3 软件设计（HAL 库）

4.1.3.1 STM32CubeMX 配置



4.1.3.2 添加其他代码

1) rtc.c 中添加代码

```
/* USER CODE BEGIN 1 */
void Alarm_SetXSecond(uint32_t x)
{
    /* 获取当前时间 */
    RTC_TimeTypeDef timeTypeDef = {0};
    HAL_RTC_GetTime(&hrtc, &timeTypeDef, RTC_FORMAT_BIN);

    /* 设置闹钟 */
    RTC_AlarmTypeDef sAlarm = {0};
    sAlarm.AlarmTime.Hours = timeTypeDef.Hours;
    sAlarm.AlarmTime.Minutes = timeTypeDef.Minutes;
    sAlarm.AlarmTime.Seconds = timeTypeDef.Seconds + x - 1;
    HAL_RTC_SetAlarm(&hrtc, &sAlarm, RTC_FORMAT_BIN);
}
/* USER CODE END 1 */
```

2) main.c

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_RTC_Init();
    MX_USART1_UART_Init();

    // 检测复位来源。 当从待机恢复的时候，会重新开始执行 main 函数
```

```

if (__HAL_PWR_GET_FLAG(PWR_FLAG_SB) == SET) // 从待机恢复
{
    // 清除待机标志位
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB);
    // 清除唤醒标记
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
}
while (1)
{
    printf("正常代码执行完毕,5s 后进入待机模式... \r\n");
    HAL_Delay(5000);
    printf("开始进入待机模式, 20s 后被唤醒... \r\n");
    // 1. 设置闹钟: 20s 后到时
    Alarm_SetXSecond(20);
    // 2. 停止滴答定时器
    HAL_SuspendTick();
    // 3. 进入待机模式
    HAL_PWR_EnterSTANDBYMode();
}
}

```

4.2 RTC 案例 2：实时时钟（掉电不丢失）

4.2.1 需求描述

显示时间。通过串口把时间发送给电脑显示。即使关机很多天，再启动后也能正确显示时间。

4.2.2 软件设计（寄存器）

4.2.2.1 main.c

```

#include "stm32f10x.h" // Device header
#include "usart.h"
#include "Delay.h"
#include "led.h"
#include "rtc_datetime.h"

int main(void)
{
    usart1_init();
    printf("尚硅谷实时时钟实验.....\r\n");
    RTC_Init();

    /* 在 window 的 powershell 获取 unix 时间戳命令: [int][double]::Parse((Get-Date -
    UFormat %s)) */
}

```

```

/* 设置一次就行了，以后时钟就可以正常运行了 */
//RTC_SetUnixTimestampSecond(1700478670);

DateTimeDefType dateTime;
while (1)
{
    RTC_GetDateTime(&dateTime);

    printf("%04d 年%02d 月%02d 日 %02d:%02d:%02d \r\n",
           dateTime.year,
           dateTime.month,
           dateTime.day,
           dateTime.hour,
           dateTime.mintue,
           dateTime.second);

    Delay_s(1);
}
}

```

4.2.2.2 rtc_datetime.h

```

#ifndef __RTC_DATETIME_C
#define __RTC_DATETIME_C

#include "stm32f10x.h"
#include "time.h"

typedef struct
{
    uint16_t year;
    uint8_t month;
    uint8_t day;
    uint8_t hour;
    uint8_t mintue;
    uint8_t second;
} DateTimeDefType;

void RTC_Init(void);
void RTC_SetUnixTimestampSecond(uint32_t unixTimestampSecond);
void RTC_GetDateTime(DateTimeDefType *dateTime);
#endif

```

4.2.2.3 rtc_realtime.c

```

#include "rtc_datetime.h"

/**

```

```

* @description: rtc 的初始化
* @return {*}
*/
void RTC_Init(void)
{
    /* 1. 使能 PWR 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_PWREN;
    RCC->BDCR &= ~RCC_BDCR_RTCEN;

    /*
        2. 使用 LSE(外部低速时钟) 作为 RTC 时钟
        00: 无时钟;
        01: LSE 振荡器作为 RTC 时钟 32.768KHz;
        10: LSI 振荡器作为 RTC 时钟;
        11: HSE 振荡器在 128 分频后作为 RTC 时钟。
    */
    RCC->BDCR |= RCC_BDCR_RTCSEL_1;
    RCC->BDCR |= RCC_BDCR_RTCSEL_0;

    /* 3. 使能 BKP 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_BKPEN;

    /*
        4. 允许向 RTC 寄存器和后备寄存器写入数据 PWR_CR_DBP 位
        0: 禁止写入 RTC 和后备寄存器
        1: 允许写入 RTC 和后备寄存器
    */
    PWR->CR |= PWR_CR_DBP;

    /* 5. 使能 RTC 时钟 */
    RCC->BDCR |= RCC_BDCR_RTCEN;

    /* 6. 使能 LSE */
    RCC->BDCR |= RCC_BDCR_LSEON;

    /* 7. 等待 RTC 进入初始化状态(上次的写操作已经完成) */
    while (!(RTC->CRL & RTC_CRL_RTOFF))
    ;

    /* 8. 进入配置模式. 允许配置 RTC 的计数器, 预分频系数和闹钟寄存器。 ----- */
    RTC->CRL |= RTC_CRL_CNF;

    /*
        9. 设置 RTC 预分频系数: 我们产生一个 1s 的信号, 让计数器每 s 加 1
    */
}

```

```

        时钟频率是 32768Hz， 所以预分频系数应该是 32768-1=0x7FFF
    */
    RTC->PRLH = 0x0;
    RTC->PRLL = 0x7FFF;

    /* 10. 退出配置模式 。前面的数据才会真正更新到寄存器中 -----*/
    RTC->CRL &= ~RTC_CRL_CNF;
}

/**
 * @description: 把计数器的值设置位 unix 时间戳
 */
void RTC_SetUnixTimestampSecond(uint32_t unixTimestampSecond)
{
    /* 1. 等待 RTC 进入初始化状态 */
    while (!(RTC->CRL & RTC_CRL_RTOFF))
        ;

    /* 2. 进入配置模式 */
    RTC->CRL |= RTC_CRL_CNF;

    /* 3. 等待秒标志位 */
    while (!(RTC->CRL & RTC_CRL_SECF))
        ;

    /* 4. 设置位 unix 时间戳*/
    RTC->CNTH = unixTimestampSecond >> 16;
    RTC->CNTL = unixTimestampSecond & 0xFFFF;

    /* 5. 退出配置模式 */
    RTC->CRL &= ~RTC_CRL_CNF;
}

/**
 * @description: 获取年月日时分秒
 * @param {DateTimeDefType} *dateTime 年月日时分秒会存储到这个结构体中
 */
void RTC_GetDateTime(DateTimeDefType *dateTime)
{
    /* 1. 等待寄存器同步 */
    while (!(RTC->CRL & RTC_CRL_RSF))
        ;

    /* 2. 读取计数器中的值 */

```

```

uint32_t second = RTC->CNTH << 16 | RTC->CNTL;

/* 3. 根据传入的秒得到一个时间结构体 */
struct tm *timeinfo = localtime(&second);

/* 年是由1900开始计算的两位年份，所以需要再加个1900 */
dateTime->year = timeinfo->tm_year + 1900;
/* 月份是从0开始，所以+1 */
dateTime->month = timeinfo->tm_mon + 1;

dateTime->day = timeinfo->tm_mday;

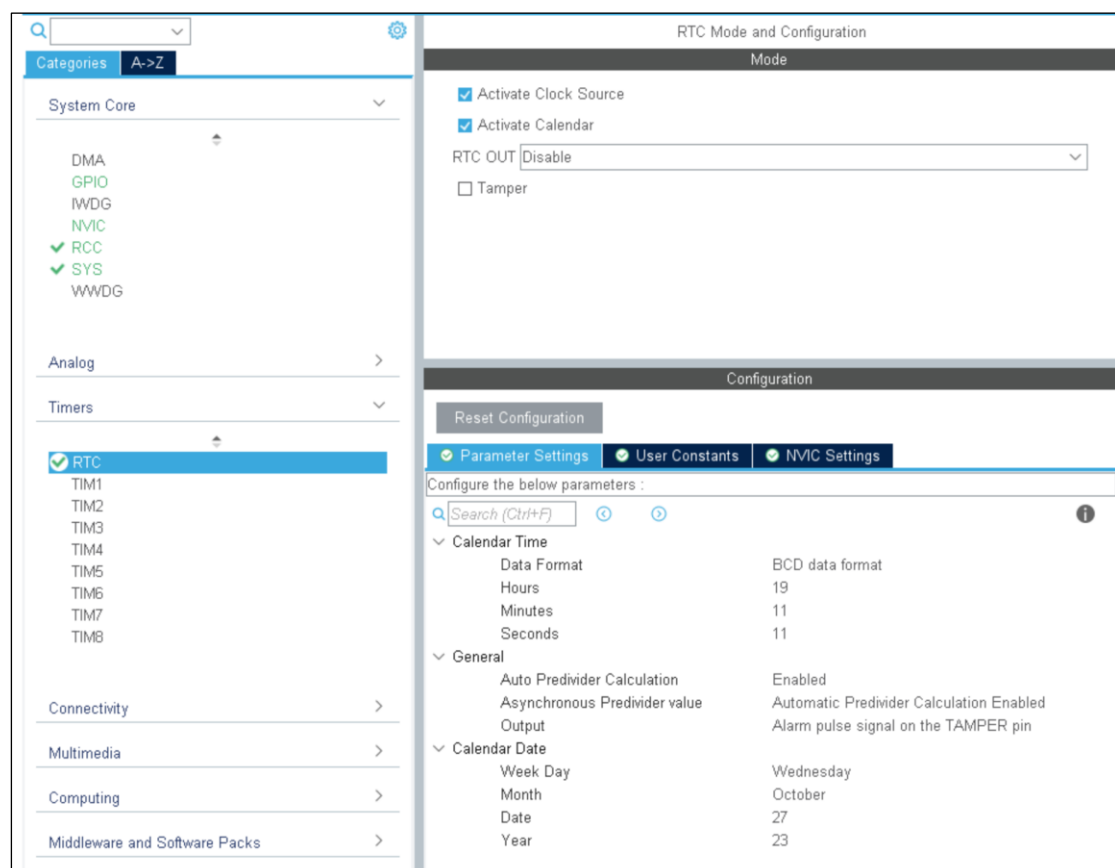
dateTime->hour = timeinfo->tm_hour;
dateTime->mintue = timeinfo->tm_min;
dateTime->second = timeinfo->tm_sec;
}

```

4.2.3 软件设计（HAL 库）

复制上个项目，适当做些修改。

4.2.3.1 STM32CubeMX 设置



4.2.3.2 添加其他代码

```
int main(void)
```

```

{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_RTC_Init();
    MX_USART1_UART_Init();
    RTC_DateTypeDef dateTypeDef;
    RTC_TimeTypeDef timeTypeDef;
    while (1)
    {
        // 一定要先获取时间，再获取日期。 获取时间的时候会对日期进行操作，比如增加一天
        HAL_RTC_GetTime(&hrtc, &timeTypeDef, RTC_FORMAT_BIN);
        HAL_RTC_GetDate(&hrtc, &dateTypeDef, RTC_FORMAT_BIN);
        printf("20%02d 年%02d 月%02d 日 %02d:%02d:%02d \r\n",
            dateTypeDef.Year,
            dateTypeDef.Month,
            dateTypeDef.Date,
            timeTypeDef.Hours,
            timeTypeDef.Minutes,
            timeTypeDef.Seconds);
        HAL_Delay(1000);
    }
}

```

第 5 章 看门狗

5.1 什么是看门狗

看门狗是一种**计时硬件电路**。

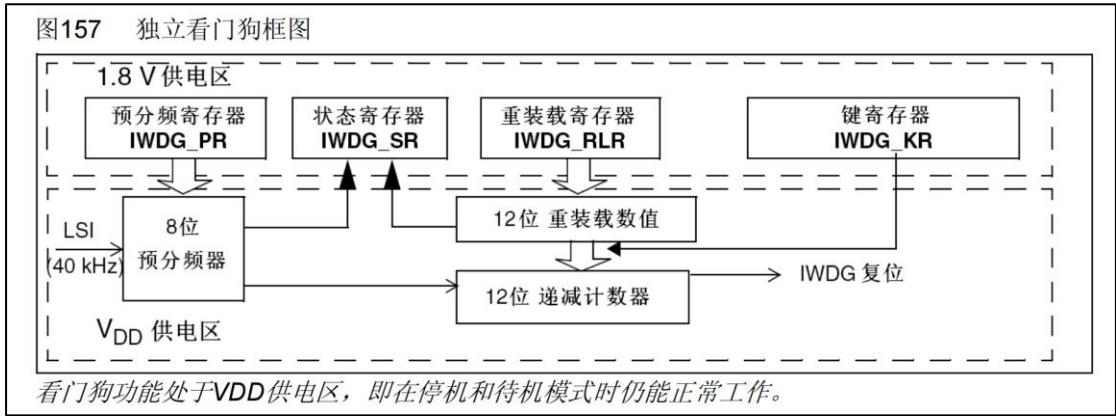
当系统环境比较恶劣复杂的时候，程序可能会出现一些不可预料的错误，导致程序卡死，崩溃等情况。看门狗就可以及时的重置程序，使程序重新从头开始运行。

5.2 STM32 的看门狗

STM32 有 2 个看门狗，**独立看门狗**和**窗口看门狗**。独立看门狗号称宠物狗，窗口看门狗号称警犬。

5.2.1 独立看门狗

IWDG（Independent watchdog，独立看门狗）。独立看门狗用通俗一点的话来解释就是一个 **12 位**的**递减**计数器，当计数器的值从某个值一直减到 0 的时候，系统就会产生一个复位信号，即 IWDG_RESET。如果在计数没减到 0 之前，刷新了计数器的值的话，那么就不会产生复位信号，这个刷新计数器值的动作就是我们经常说的喂狗。



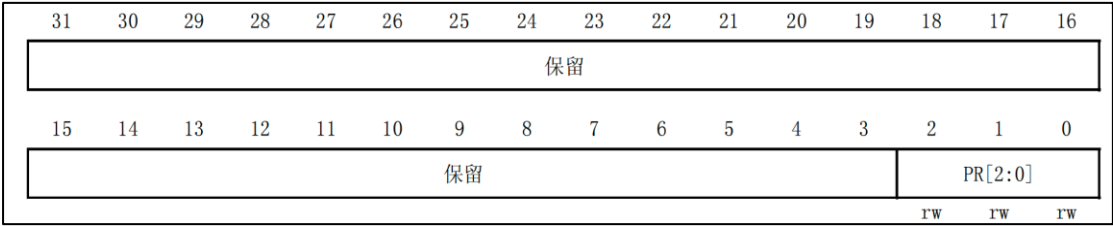
1) 计数器时钟

独立看门狗的时钟由独立的 RC 振荡器 **LSI** 提供，即使主时钟发生故障它仍然有效，非常独立（叫独立看门狗的由来）。

LSI 的频率一般在 30~60KHZ 之间，根据温度和工作场合会有一定的漂移，我们一般取 40KHZ，所以独立看门狗的定时时间并不一定非常精确，只适用于对时间精度要求比较低的场合。

2) 预分频寄存器

这里存储着我们想要的预分频值。



PR[2:0]: 预分频因子 (Prescaler divider)

这些位具有写保护设置，参见17.3.2节。通过设置这些位来选择计数器时钟的预分频因子。要改变预分频因子，IWDG_SR寄存器的PVU位必须为0。

000: 预分频因子=4	100: 预分频因子=64
001: 预分频因子=8	101: 预分频因子=128
010: 预分频因子=16	110: 预分频因子=256
011: 预分频因子=32	111: 预分频因子=256

注意：对此寄存器进行读操作，将从VDD电压域返回预分频值。如果写操作正在进行，则读回的值可能是无效的。因此，只有当IWDG_SR寄存器的PVU位为0时，读出的值才有效。

3) 12 位递减计数器

独立看门狗的计数器是一个 12 位的递减计数器，最大值为 0XFFF，当计数器减到 0 时，会产生一个复位信号:IWDG_RESET，让程序重新启动运行，如果在计数器减到 0 之前刷新了计数器的值的话，就不会产生复位信号，重新刷新计数器值的这个动作我们俗称喂狗。

4) 重载寄存器

重载寄存器是一个 12 位的寄存器，里面装着要刷新到计数器的值，这个值的大小决定着独立看门狗的溢出时间。

表83 看门狗超时时间(40kHz的输入时钟(LSI)) ⁽¹⁾			
预分频系数	PR[2:0]位	最短时间(ms) RL[11:0] = 0x000	最长时间(ms) RL[11:0] = 0xFFFF
/4	0	0.1	409.6
/8	1	0.2	819.2
/16	2	0.4	1638.4
/32	3	0.8	3276.8
/64	4	1.6	6553.6
/128	5	3.2	13107.2
/256	(6或7)	6.4	26214.4

这些时间是按照40kHz时钟给出。实际上，MCU内部的RC频率会在30kHz到60kHz之间变化。此外，即使RC振荡器的频率是精确的，确切的时序仍然依赖于APB接口时钟与RC振荡器时钟之间的相位差，因此总会有一个完整的RC周期是不确定的。

这里的超时时间是指如果超过这个时间不喂狗，就会产生复位信号。超时时间的计算公式：

式： $T_{IWDG} = \frac{1}{\text{时钟频率}} * \text{预分频系数} * (RL + 1)$

注意：

(1) 时钟频率是 40KHz。

(2) 预分频系数只有表中固定的这些值（3 位 PR 值决定）。

(3) RL 的值就是重装载寄存器中的值。计数器的初始值。

5) 键寄存器

键寄存器 IWDG_KR 独立看门狗的一个控制寄存器，只能写入 0-15 位，读永远是 0。三种控制方式，往这个寄存器写入下面三个不同的值有不同的效果。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY[15:0]															
W	W	W	W	W	W	W	W	W	W	W	W	W	W	W	W

写入键寄存器的值	作用
0xCCCC	软件启动看门狗。若启用了硬件看门狗，则此命令无效。看门狗一旦启动，无法停止。
0xAAAA	重装载计数器的值加入到计数器中。（喂狗）
0x5555	允许访问 IWDG_PR（预分频计数器）和 IWDG_RLR（重装载计数器）寄存器。
其他值	开启 IWDG_PR（预分频计数器）和 IWDG_RLR（重装载计数器）寄存器的写保护。

6) 状态寄存器

状态寄存器 SR 只有位 0: PVU 和位 1: RVU 有效，这两位只能由硬件操作，软件操作不了。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留														RVU	PVU
														r	r
位31:2		保留。													
位1		RVU: 看门狗计数器重装载值更新 (Watchdog counter reload value update) 此位由硬件置'1'用来指示重装载值的更新正在进行中。当在VDD域中的重装载更新结束后，此位由硬件清'0'(最多需5个40kHz的RC周期)。重装载值只有在RVU位被清'0'后才可更新。													
位0		PVU: 看门狗预分频值更新 (Watchdog prescaler value update) 此位由硬件置'1'用来指示预分频值的更新正在进行中。当在VDD域中的预分频值更新结束后，此位由硬件清'0'(最多需5个40kHz的RC周期)。预分频值只有在PVU位被清'0'后才可更新。													

5.2.2 窗口看门狗

[illegible]

3) 计数器

18.6.1 控制寄存器(WWDG_CR)																													
地址偏移量: 0x00																													
复位值: 0x7F																													
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	保留													
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	保留													
保留								WDGA	T6	T5	T4	T3	T2	T1	T0														
								r's	r'w	r'w	r'w	r'w	r'w	r'w	r'w														

窗口看门狗的计数器是一个递减计数器,共有 7 位,其值存在控制寄存器 CR 的 T[6:0] 位,当 7 个位全部为 1 时是 0X7F,这个是最大值。

当递减到 T6 位变成 0 时,即从 0X40 变为 0X3F 时候,会产生看门狗复位。这个值 0X40 是看门狗能够递减到的最小值,所以计数器的值只能是: 0X40~0X7F 之间。

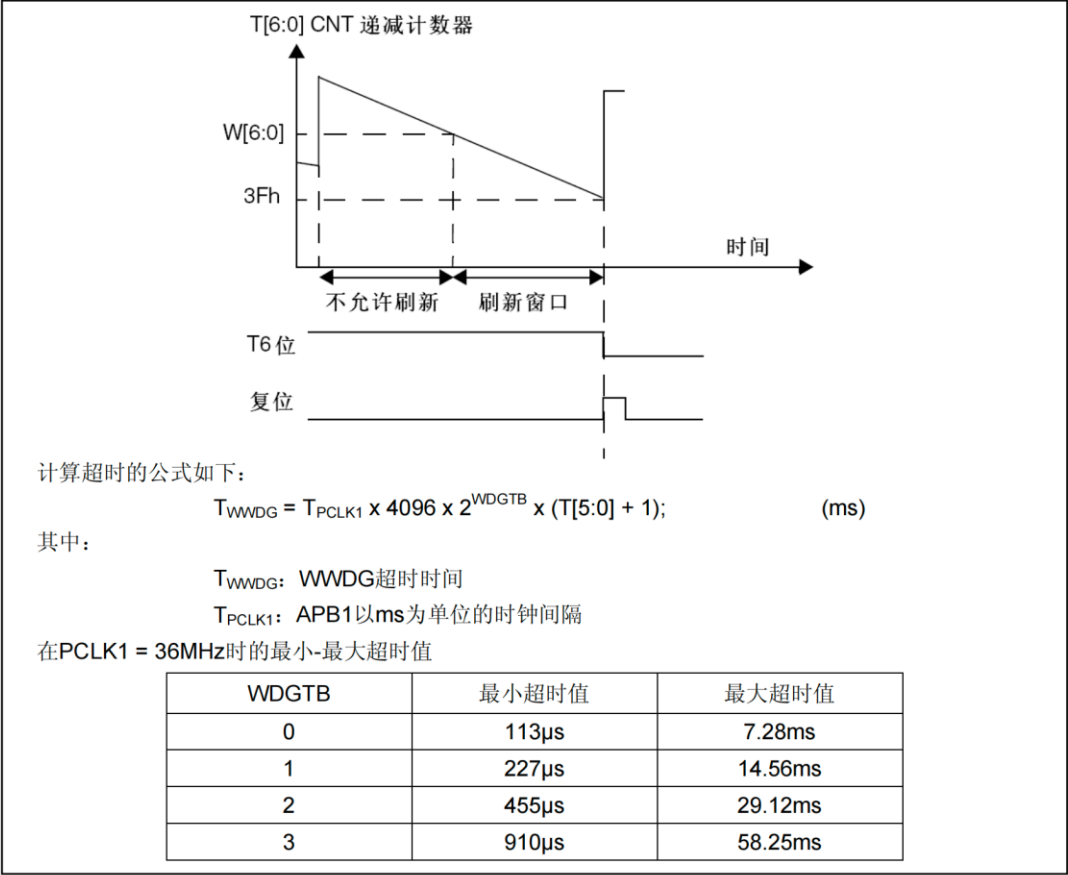
实际上真正用来计数的是 T[5:0]。当递减计数器递减到 0X40 的时候,还不会马上产生复位,如果使能了提前唤醒中断: CFR 位 9EWI 置 1,则产生提前唤醒中断,如果真进入了这个中断的话,就说明程序肯定是出问题了,那么在中断服务程序里面我们就需要做最重要的工作,比如保存重要数据,或者报警等,这个中断我们也叫它**死前中断**。

4) 窗口值

窗口看门狗必须在计数器的值在一个**范围**内才**可以喂狗**,其中下窗口的值是固定的 0X40,上窗口的值可以改变,具体的由配置寄存器 CFR 的位 W[6:0] 设置。其值必须大于 0X40,如果小于或者等于 0X40 就是失去了窗口的价值,而且也不能大于计数器的值,所以必须得小于 0X7F。

那窗口值具体要设置成多大? 这个得根据我们需要监控的程序的运行时间来决定。如果我们要监控的程序段 A 运行的时间为 Ta,当执行完这段程序之后就要进行喂狗,如果在窗口时间内没有喂狗的话,那程序就肯定是出问题了。一般计数器的值设置成最大 0x7F,窗口值为 WR,计数器减一个数的时间为 T,那么时间: (0x7F-WR)*T 应该稍微大于 Ta 即可,这样就能做到刚执行完程序段 A 之后喂狗,起到监控的作用,这样也就可以算出 WR 的值是多少。

图159 窗口看门狗时序图



5.2.3 独立看门狗和窗口看门口的对比

	独立看门狗	窗口看门狗
复位	计数减到 0	计数器 T[5,0]减到 0 和过早重装计数器
喂狗方式	键寄存器写入 0xAAAA，重装固定值 RLR	直接写入寄存器，写多少重装多少
中断	无	死前中断
时钟源	LSI（40KHz）	PCLK1（36MHz）
预分频系数	4，8，32，64，128，256	1，2，4，8
计数器	12 位	6 位
超时时间	0.1ms 到 26214.4ms	113us 到 58.25ms
用途	独立工作，对时间精度要求比较低	要求看门狗在计时窗口起作用

5.3 独立看门狗案例

5.3.1 需求描述

验证独立看门狗是否能复位程序。假设程序正常执行需要 3s，执行完毕之后喂狗，我们把超时时间设置为 4s。当检测到按键 1 按下后，我们延长 3s，那么喂狗超时，就会产生复位。

5.3.2 软件设计（寄存器）



看门狗案例用到的
寄存器详解.docx

5.3.2.1 main.c

```
#include "stm32f10x.h" // Device header
#include "usart.h"
#include "Delay.h"
#include "key.h"
#include "iwdg.h"

int8_t flag = 0;
int main(void)
{
    usart1_init();
    key_init();
    printf("尚硅谷独立看门狗实验.....\r\n");
    IWDG_Init();
    while (1)
    {
        printf("程序正常执行.....\r\n");
        Delay_s(3);
        if (flag)
        {
            // 按键按下，模拟程序执行时间加长
            Delay_s(4); // 延迟 4s，会导致喂狗不及时，程序会自动复位
            flag = 0;
        }
        IDWDG_Refresh();
    }
}

/* 按键按下的中断服务函数 */
void EXTI15_10_IRQHandler(void)
{
    printf("你按下了按键.....\r\n");
```

```

    flag = 1;
    /* 清除中断标志 */
    EXTI->PR |= EXTI_PR_PR10;
}

```

5.3.2.2 iwdg.h

```

#ifndef __IWDG_H
#define __IWDG_H
#include "stm32f10x.h"

void IWDG_Init(void);

void IDWDG_Refresh(void);

#endif

```

5.3.2.3 iwdg.c

```

#include "iwdg.h"

/**
 * @description: 独立看门狗的初始化
 * @return {*}
 */
void IWDG_Init(void)
{
    /* 1. 启动看门狗.看门狗一旦启动就无法停止. kR=0xCCCC 会自动开启 LSI */
    IWDG->KR = 0xCCCC;

    /* 2. 允许访问预分频寄存器和重装载寄存器 KR=0x5555 */
    IWDG->KR = 0x5555;

    /* 3. 写入预分频值 100:64 分频 */
    IWDG->PR = 4;

    /* 4. 写入重装载值 */
    IWDG->RLR = 2499;

    /* 5. 初始化完成之前用重装载值更新下计数器的值（喂狗） */
    IDWDG_Refresh();
}

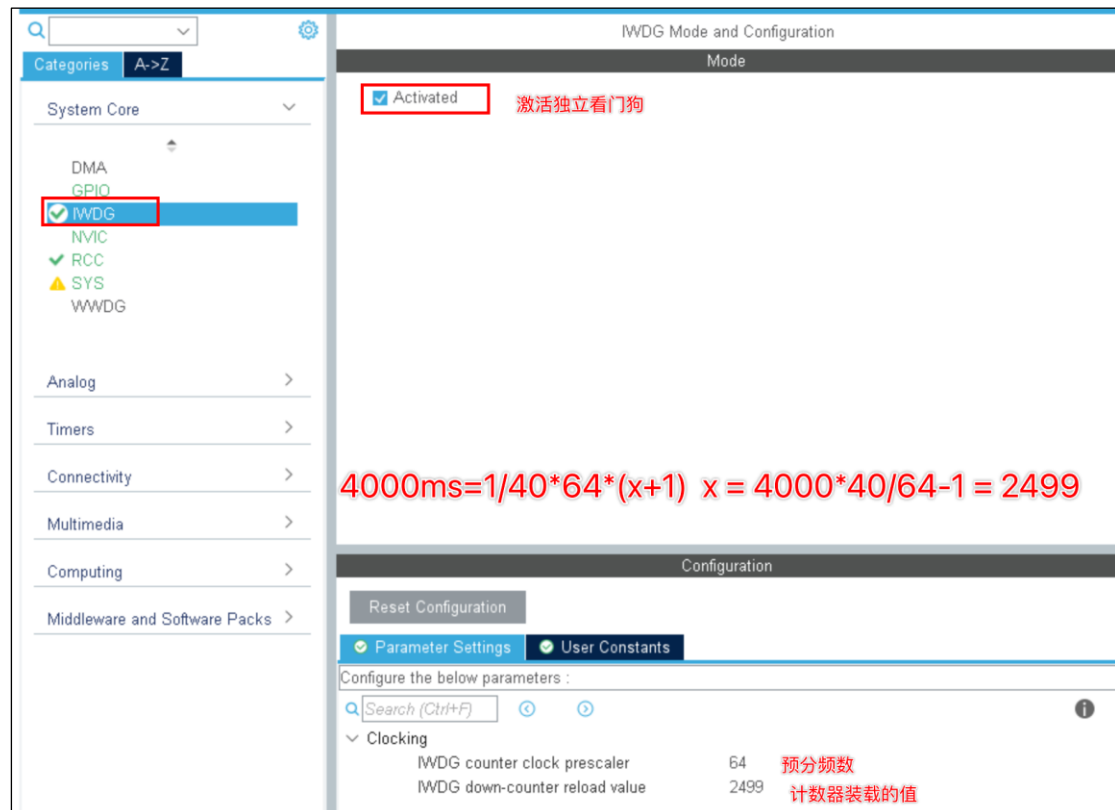
/**
 * @description: 喂狗
 * @return {*}
 */

```

```
void IDWDG_Refresh(void)
{
    /* 更新计数器的值 KR=0xAAAA */
    IWDG->KR = 0xAAAA;
}
```

5.3.3 软件设计（HAL 库）

5.3.3.1 STM32CubeMX 配置



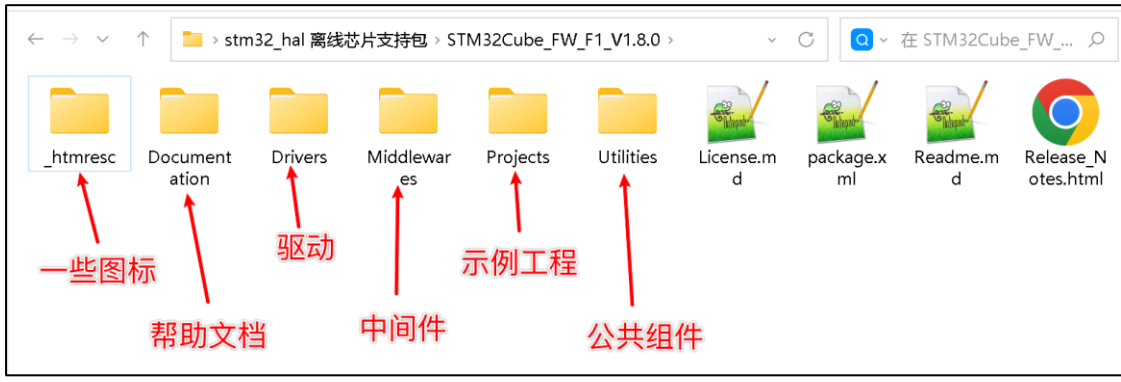
5.3.3.2 添加其他代码

```
int flag = 0;
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_IWDG_Init();
    MX_USART1_UART_Init();
    printf("程序复位开始运行...\r\n");
    while (1)
    {
        printf("要监控的程序开始...\r\n");
        HAL_Delay(3000); // 执行 3s
        if (flag)
```

```
    {
        // 如果按键按下
        HAL_Delay(4000); // 这里延时 4s 导致喂狗不及时，程序会自动复位
    }
    HAL_IWDG_Refresh(&hiwdg); // 喂狗
    printf("要监控的程序执行结束... \r\n");
}
}
// 按键按下的中断回调函数
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    printf("你按下了按键\r\n");
    flag = 1;
}
```

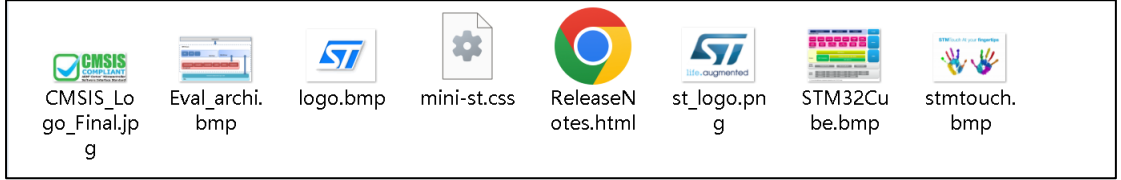
第 6 章 附录 1: HAL 库代码分析

6.1 HAL 库目录结构



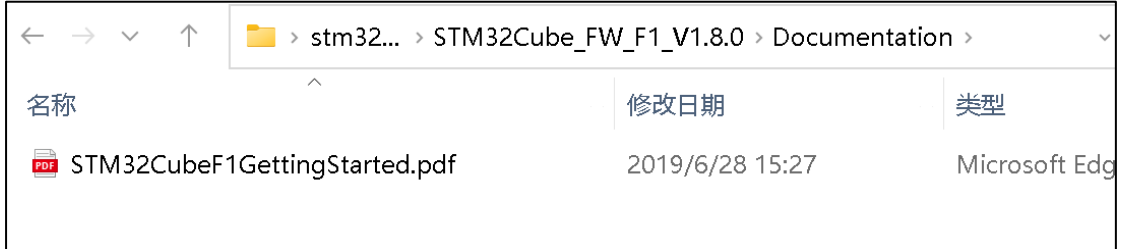
(1) _htmresc

下面存储了一些图标。



(2) Documentation

里面存储了 ST 官方提供的 HAL 的使用手册。



(3) Drivers

里面存储了 CMISIS 库，HAL 库（片内外设驱动），官方开发板的板载外设驱动。



(4) Middlewares

中间件，包含 ST 官方的 STemWin、STM32_Audio、STM32_USB_Device_Library、

STM32_USB_Host_Library；也有第三方的 fatfs 文件系统等等。

← → ▾ ▴				> stm32_ha... > STM32Cube_FW_F1_V1.8.0 > Middlewares >		🔍 在 Middlewares 中搜索 🔍	
名称				修改日期		类型	大小
📁 ST				2023/9/28 11:21		文件夹	
📁 Third_Party				2023/9/28 11:22		文件夹	

(5) Projects

文件夹下是用驱动库写的针对官方发行 demo 板的例子和工程模板。

← → ▾ ▴				> stm32_hal 离... > STM32Cube_FW_F1_V1.8.0 > Projects >		🔍 在 Projects 中搜索 🔍	
名称				修改日期		类型	大小
📁 STM32F103RB-Nucleo				2019/7/3 10:10		文件夹	
📁 STM32VL-Discovery				2019/7/3 10:10		文件夹	
📁 STM3210C_EVAL				2023/9/28 11:52		文件夹	
📁 STM3210E_EVAL				2019/7/3 10:11		文件夹	
🌐 STM32CubeProjectsList.html				2019/5/9 12:10		Chrome HTML Doc...	82 KB

(6) Utilities

实用的公用组件比如 LCD_LOG 实用液晶打印调试信息等。

← → ▾ ▴				> stm32_hal 离线... > STM32Cube_FW_F1_V1.8.0 > Utilities >		🔍 在 Utilities 中搜索 🔍	
名称				修改日期		类型	大小
📁 CPU				2019/6/28 15:27		文件夹	
📁 Fonts				2019/6/28 15:27		文件夹	
📁 Log				2019/6/28 15:27		文件夹	
📁 Media				2019/6/28 15:27		文件夹	

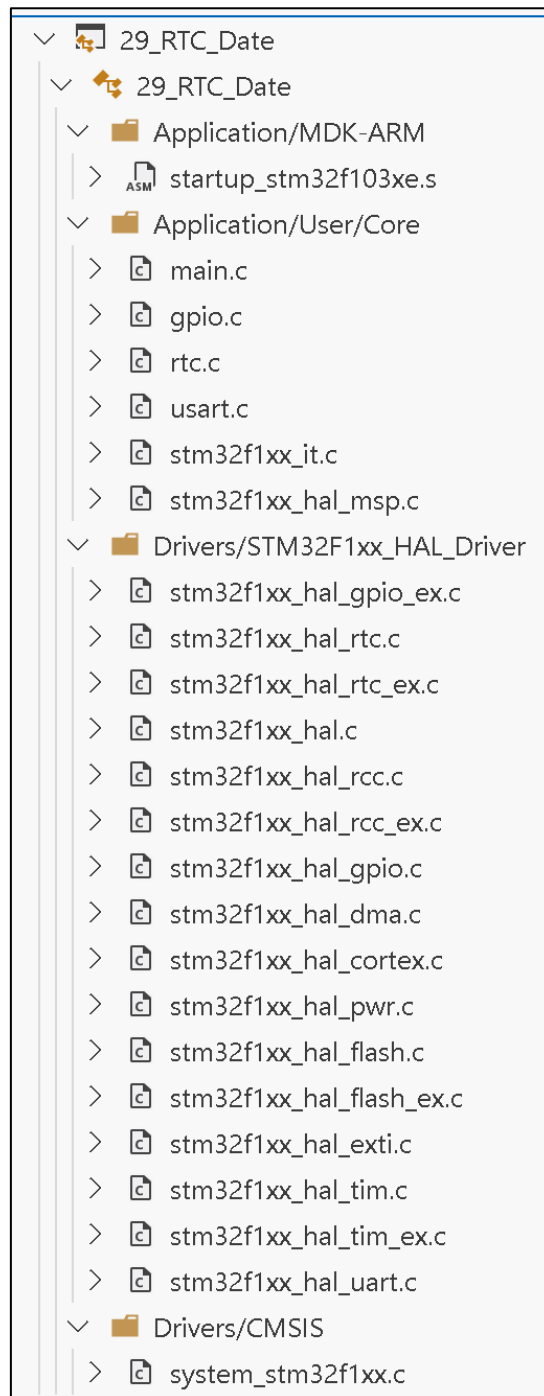
6.2 使用 STM32CubeMX 创建项目

在使用 STM32CubeMX 创建项目的时候，会自动把用到的一些文件 Copy 到我们的项目中。非常的方便不用手动 Copy。

Project	STM32Cube MCU packages and embedded software packs	
	☐	Copy all used libraries into the project folder
	☒	Copy only the necessary library files
	☐	Add necessary library files as reference in the toolchain project configuration file

6.3 项目中主要文件的作用

我们以一个项目为例，说下几个主要文件的功能。



6.3.1 startup_stm32f103xe.s

这个是程序的启动文件，STM32CubeMX 根据你选择的新品会自动选择对用的启动文件（不同的新品启动文件不一样）。这个是有 ST 官方实现的，我们无需做任何改变。

内部定义有中断向量表，当我们需要写中断响应函数的时候，可以从启动文件中查找。

6.3.2 system_stm32f1xx.c

这个文件包含了 STM32 芯片上电后扩展外部存储器用的函数。例如 SystemInit 函数，用于上电后扩展外部 SRAM，该函数的定义就存储在 system_stm32f1xx.c 文件。

STM32F103 系列的芯片，以前在库函数版本中调用这个 SystemInit 函数后，系统时钟被初始化为 72MHz。现在时钟的初始化已经专门放在了其他地方。

6.3.3 stm32f1xx_it.c

这个文件是专门用来编写**中断服务函数**。不过在使用 HAL 的时候，一般使用 HAL 库提供的中断回调函数更方便。后面再细说。

```
void EXTI0_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI0_IRQn 0 */

    /* USER CODE END EXTI0_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(KEY1_Pin);
    /* USER CODE BEGIN EXTI0_IRQn 1 */

    /* USER CODE END EXTI0_IRQn 1 */
}

void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
    /* EXTI line interrupt detected */
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
    {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
        HAL_GPIO_EXTI_Callback(GPIO_Pin);
    }
}

// 我们只需完成下面的回调函数即可。这个函数是一个 weak 修饰的函数，如果在别的地方有完整实现，则这里的就忽略了。
__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
}
```

6.3.4 stm32f1xx_hal_conf.h

支持所有 STM32F1 型号的芯片，但有的型号芯片外设功能比较多，有的少，使用这个配置文件根据芯片型号增减 ST 库的外设文件。

```
#define HAL_MODULE_ENABLED
/*#define HAL_ADC_MODULE_ENABLED */
/*#define HAL_CRYP_MODULE_ENABLED */
/*#define HAL_CAN_MODULE_ENABLED */
/*#define HAL_CAN_LEGACY_MODULE_ENABLED */
/*#define HAL_CEC_MODULE_ENABLED */
/*#define HAL_CORTEX_MODULE_ENABLED */
/*#define HAL_CRC_MODULE_ENABLED */
/*#define HAL_DAC_MODULE_ENABLED */
/*#define HAL_DMA_MODULE_ENABLED */
/*#define HAL_ETH_MODULE_ENABLED */
```

```
/*#define HAL_FLASH_MODULE_ENABLED */
#define HAL_GPIO_MODULE_ENABLED
.....
```

在使用 STM32CubeMX 生成代码的时候，已经替我们增减好了，我们自己基本不需要做什么。

6.3.5 stm32f1xx_hal.c

HAL 中很重要的文件。HAL 的初始化函数，反初始化函数等都是在这里定义的。

```
HAL_StatusTypeDef HAL_Init(void);
HAL_StatusTypeDef HAL_DeInit(void);
void HAL_MspInit(void);
void HAL_MspDeInit(void);
HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority);
.....
```

外设控制函数。

```
void HAL_IncTick(void);
void HAL_Delay(uint32_t Delay);
uint32_t HAL_GetTick(void);
uint32_t HAL_GetTickPrio(void);
HAL_StatusTypeDef HAL_SetTickFreq(HAL_TickFreqTypeDef Freq);
HAL_TickFreqTypeDef HAL_GetTickFreq(void);
void HAL_SuspendTick(void);
void HAL_ResumeTick(void);
uint32_t HAL_GetHalVersion(void);
uint32_t HAL_GetREVID(void);
uint32_t HAL_GetDEVID(void);
uint32_t HAL_GetUIDw0(void);
uint32_t HAL_GetUIDw1(void);
uint32_t HAL_GetUIDw2(void);
void HAL_DBGMCU_EnableDBGSleepMode(void);
void HAL_DBGMCU_DisableDBGSleepMode(void);
void HAL_DBGMCU_EnableDBGStopMode(void);
void HAL_DBGMCU_DisableDBGStopMode(void);
void HAL_DBGMCU_EnableDBGStandbyMode(void);
void HAL_DBGMCU_DisableDBGStandbyMode(void);
.....
```

6.4 main 函数的执行

使用 STM32Cube 生成的代码中已经自动帮我们调用好了一些初始化函数。

```
/* Reset of all peripherals, Initializes the Flash interface and
the Systick.
重置所有外设，初始化 flash 接口和系统滴答时钟
*/
HAL_Init();

/* Configure the system clock
配置系统时钟
*/
SystemClock_Config();

/* Initialize all configured peripherals
初始化你配置的所有外设
*/
```

```

*/
MX_GPIO_Init();
MX_RTC_Init();
MX_USART1_UART_Init();
.....

```

6.4.1 HAL_Init()

```

HAL_StatusTypeDef HAL_Init(void)
{
    // 为方便理解，精简了一些代码
    /* Set Interrupt Group Priority 设置中断优先级组*/
    HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4);

    /* Use systick as time base source and configure 1ms tick (default
    clock after Reset is HSI)
    初始化滴答时钟
    */
    HAL_InitTick(TICK_INT_PRIORITY);

    /* Init the low level hardware
    初始化底层硬件
    */
    HAL_MspInit();

    /* Return function status */
    return HAL_OK;
}

```

它用于初始化 HAL 库，在 main 函数中必须先执行。主要做了这几件事：配置 flash 读取，配置系统滴答时钟（每 1ms 产生一次中断），配置 NVIC 优先级组为 4，最后调用 HAL_MspInit()去初始化底层硬件。

其中：HAL_MspInit() 这个是每项目的实现是不一样的，不过这个一般不需要时序，因为每个外设都会定义自己的 HAL_xxx_MspInit。

```

200  __weak void HAL_MspInit(void)
201  {
202      /* NOTE : This function should not be modified, when the callback is needed,
203         |         the HAL_MspInit could be implemented in the user file
204         */
205  }

```

6.4.2 SystemClock_Config()

这个用来配置系统时钟。在 STM32CubeMX 中配置好之后，代码会自动生成，方法的实现位置在 main.c 文件中。

```

void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};
}

```

```

    /** Initializes the RCC Oscillators according to the specified
parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE |
RCC_OSCILLATORTYPE_LSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.LSEState = RCC_LSE_ON;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct,
FLASH_LATENCY_2) != HAL_OK)
    {
        Error_Handler();
    }
    PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_RTC;
    PeriphClkInit.RTCClockSelection = RCC_RTCCCLKSOURCE_LSE;
    if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
    {
        Error_Handler();
    }
}

```

6.4.3 MX_GPIO_Init

这个是对 GPIO 的初始化，也是根据你的配置自动生成的。这个放在文件 `gpio.c` 中。

```

void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /*Configure GPIO pin : PtPin */
    GPIO_InitStruct.Pin = KEY1_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(KEY1_GPIO_Port, &GPIO_InitStruct);
}

```

```

/* EXTI interrupt init*/
HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(EXTI0_IRQn);

}

```

6.4.4 MX_USART1_UART_Init

这个是串口的初始化。在 `usart.c` 文件中。

```

void MX_USART1_UART_Init(void)
{

    /* USER CODE BEGIN USART1_Init 0 */

    /* USER CODE END USART1_Init 0 */

    /* USER CODE BEGIN USART1_Init 1 */

    /* USER CODE END USART1_Init 1 */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart1.Init.OverSampling = UART_OVERSAMPLING_16;
    // 初始化 UART, 内部会调用 HAL_UART_MspInit
    if (HAL_UART_Init(&huart1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART1_Init 2 */

    /* USER CODE END USART1_Init 2 */

}
// 对 uart 用到的硬件的初始化
void HAL_UART_MspInit(UART_HandleTypeDef* uartHandle)
{

    GPIO_InitTypeDef GPIO_InitStruct = {0};
    if(uartHandle->Instance==USART1)
    {
        /* USER CODE BEGIN USART1_MspInit 0 */

        /* USER CODE END USART1_MspInit 0 */
        /* USART1 clock enable */
        __HAL_RCC_USART1_CLK_ENABLE();

        __HAL_RCC_GPIOA_CLK_ENABLE();
        /**USART1 GPIO Configuration
        PA9      -----> USART1_TX
        PA10     -----> USART1_RX
        */
        GPIO_InitStruct.Pin = GPIO_PIN_9;
        GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;

```

```

    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /* USER CODE BEGIN USART1_MspInit 1 */

    /* USER CODE END USART1_MspInit 1 */
}
}

```

说明：像这种针对某个外设的初始化，首先是硬件无关的初始化，主要就是一些和硬件无关的参数，比如波特率，校验位等等。然后在 HAL_UART_MspIni 中对用到的硬件进行初始化。基本上每个外设都是这个套路。

6.5 HAL 编程模式

在 HAL 库中对外设模型进行了统一，一般支持三种编程方式（以 I²C 为例）：

（1）轮询模式/阻塞模式

```

HAL_I2C_Master_Transmit();
HAL_I2C_Master_Receive();
HAL_I2C_Slave_Transmit();
HAL_I2C_Slave_Receive()
HAL_I2C_Mem_Write();
HAL_I2C_Mem_Read();
HAL_I2C_IsDeviceReady()

```

（2）中断方式

```

HAL_I2C_Master_Transmit_IT();
HAL_I2C_Master_Receive_IT();
HAL_I2C_Slave_Transmit_IT()
HAL_I2C_Slave_Receive_IT();
HAL_I2C_Mem_Write_IT();
HAL_I2C_Mem_Read_IT()

```

说明：中断方式处理，一般要实现中断回调函数（接收数据）。

```

__weak void HAL_I2C_MasterRxCpltCallback(I2C_HandleTypeDef *hi2c)
{
    /* Prevent unused argument(s) compilation warning */
    UNUSED(hi2c);

    /* NOTE : This function should not be modified, when the callback
    is needed,
               the HAL_I2C_MasterRxCpltCallback could be implemented
    in the user file
    */
}

```

（3）DMA 模式（如果支持）

```

HAL_I2C_Master_Transmit_DMA();

```

```
HAL_I2C_Master_Receive_DMA();  
HAL_I2C_Slave_Transmit_DMA();  
HAL_I2C_Slave_Receive_DMA();  
HAL_I2C_Mem_Write_DMA();  
HAL_I2C_Mem_Read_DMA()
```

第 7 章 附录 2: Keil MDK 程序编译过程

7.1 普通 C 源文件到可执行的过程

预处理（Preprocessing），编译（Compilation），汇编（Assemble），链接（Linking）。

1) 预处理

以“#”号开头的预处理指令，比如包含#include,宏定义指定#define 等。在源程序中这些指令都放在函数之外，而且一般放在源文件的前面。使用预处理器把源文件（c/h）经过预处理生成 xxx.i 文件。（简单来说就是展开程序中的宏等）

2) 编译

指将经过预处理文件（xxx.i）之后的程序转换成特定汇编（xxx.s）代码的过程。

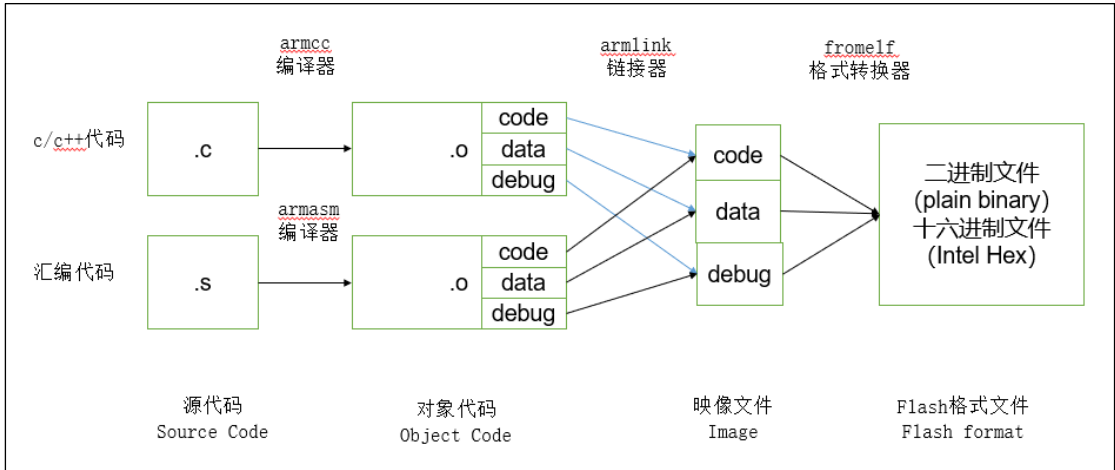
3) 汇编

汇编过程将上一步生成的汇编代码转换成机器码，这一步产生的文件叫做目标文件，是二进制格式。

4) 链接

链接过程使用链接器将上述目标文件与其它目标文件、库文件、启动文件等链接起来生成可执行文件。其他目标文件包括静态连接库和动态连接库。

7.2 Keil MDK 编译过程



- (1) 编译。Keil MDK 软件使用的编译器是 armcc 和 armasm，它们根据每个 c/c++ 和汇编源文件编译成对应的以“.o”为后缀名的对象文件 (Object Code，也称目标文件)，其内容主要是从源文件编译得到的机器码，包含了代码、数据以及调试使用的信息。
- (2) 链接。链接器 armlink 把各个.o 文件及库文件链接成一个映像文件“.axf”或“.elf”。

(3) 格式转换, 一般来说 Windows 或 Linux 系统使用链接器直接生成可执行映像文件 elf 后, 内核根据该文件的信息加载后, 就可以运行程序了, 但在单片机平台上, 需要把该文件的内容加载到芯片上, 所以还需要对链接器生成的 elf 映像文件利用格式转换器 fromelf 转换成 “.bin” 或 “.hex” 文件, 交给下载器下载到芯片的 FLASH 或 ROM 中。

```
Build Output
Rebuild started: Project: 29_RTC_Data
** Using Compiler 'VS.06 update 7 (build 960)', folder: 'C:\Keil v5\ARM\arm5 compiler\Bin'
Rebuild target '29_RTC_Data'
assembling startup_stm32f103xe.s...
compiling stm32f1xx_hal_gpio_ex.c...
compiling i2c.c...
compiling stm32f1xx_hal_msp.c...
compiling main.c...
compiling usart.c...
compiling gpio.c...
compiling rtc.c...
compiling stm32f1xx_it.c...
compiling stm32f1xx_hal_gpio.c...
compiling stm32f1xx_hal.c...
compiling stm32f1xx_hal_cortex.c...
compiling stm32f1xx_hal_pwr.c...
compiling stm32f1xx_hal_rcc_ex.c...
compiling stm32f1xx_hal_rcc.c...
compiling stm32f1xx_hal_dma.c...
compiling stm32f1xx_hal_i2c.c...
compiling stm32f1xx_hal_tim_ex.c...
compiling stm32f1xx_hal_tim.c...
compiling stm32f1xx_hal_exti.c...
compiling stm32f1xx_hal_rtc_ex.c...
compiling stm32f1xx_hal_flash.c...
compiling stm32f1xx_hal_flash_ex.c...
compiling stm32f1xx_hal_rtc.c...
compiling system_stm32f1xx.c...
compiling stm32f1xx_hal_uart.c...
linking...
Program Size: Code=7268 RO-data=396 RW-data=20 ZI-data=1204
FromELF: creating hex file...
"29_RTC_Data\29_RTC_Data.axi" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:05
```

7.3 程序组成

```
Program Size: Code=7268 RO-data=396 RW-data=20 ZI-data=1204
```

经过链接后代码分为 4 部分, 分别为: Code、RO-data、RW-data、ZI-data

(1) **Code:** 代码区。指代程序中的代码, 即函数体的大小, 这些会被存储到 ROM(Flash) 中。注意程序中未使用的函数也会算在 Code 中, 也即会占用 FLASH 空间, 因此不用的函数最好删除掉, 以免占用过多 FLASH 空间。

(2) **RO-data:** Read Only data, 即只读数据区, 它指程序中用到的只读数据, 这些数据被存储在 ROM 区, 因而程序不能修改其内容。例如 C 语言中 const 关键字定义的变量就是典型的 RO-data, 还有已经初始化的字符串等。

(3) **RW-data:** Read Write data, 即可读写数据区。特指已初始化的可读可写全局/静态变量。

初始化为“非 0 值”的可读写数据, 程序刚运行时, 这些数据具有非 0 的初始值, 且运行的时候它们会常驻在 RAM 区, 因而应用程序可以修改其内容。例如 C 语言中使用定义的全局变量, 且定义时赋予“非 0 值”给该变量进行初始化。

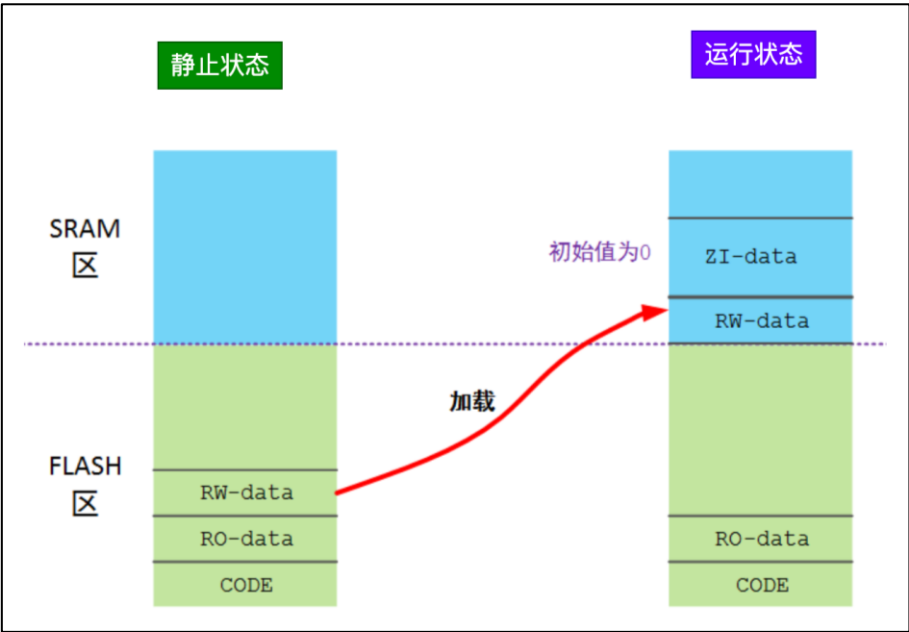
(4) **ZI-data**: Zero Initialie data, 即 0 初始化数据, 它指初始化为“0 值”的可读写数据区。它与 RW-data 的区别是程序刚运行时这些数据初始值全都为 0, 而后续运行过程与 RW-data 的性质一样, 它们也常驻在 RAM 区, 因而应用程序可以更改其内容。例如 C 语言中使用定义的全局变量, 且定义时赋予“0 值”给该变量进行初始化 (若定义该变量时没有赋予初始值, 编译器会把它当 ZI-data 来对待, 初始化为 0);

那么**局部变量**存储到什么地方呢? 在 ZI-Data 的堆栈空间中。

在 C 语言中, 函数内部定义的局部变量属于栈空间, 进入函数的时候从向栈空间申请内存给局部变量, 退出时释放局部变量, 归还内存空间。而使用 malloc 动态分配的变量属于堆空间。在程序中的栈空间和堆空间都是属于 ZI-data 区域的, 这些空间都会被初始化为 0 值。编译器给出的 ZI-data 占用的空间值中包含了堆栈的大小。

程序组件	所属类别
机器代码指令	Code
常量	RO-data
初值非 0 的全局变量	RW-data
初值为 0 的全局变量	ZI-data
局部变量	ZI-data 栈空间
使用 malloc 动态分配的空间	ZI-data 堆空间

7.4 程序的静止状态和运行状态



1) 静止状态

向 Flash 中下载程序时，其实仅仅下载的是 CODE+RO-data+RW-data 的内容，意思就是说，在掉电情况下，Flash 里面的记忆体仅包含 CODE+RO-data+ RW-data 这三块。

2) 运行状态

当上电后，程序运行时，首先程式会从特定的地址进行启动，启动时会将 RW-data 的数据载入到 SRAM 中(启用文件中有一段代码是干这件事的，ST 已经帮我们做完了这件事)。

单片机的 RO 区域不需要载入到 SRAM，内核直接从 FLASH 读取指令运行。

那 ZI-data 的数据怎么办呢？对于初始值为 0 全局变数来说，因为要在 Code 区要调用该全局变数，所以肯定要对其进行描述，程序运行时就知道了，原来你是初始值为 0 的全局变数呀，然后就在 SRAM 区给你分配了一段固定区域的地址。

对于局部变量来说，会自动分配大小，不用我们管。SRAM 中一段特定的区域是运行 ZI-data 数据，RW-DATA+ZI-DATA 就是程序运行总共会占用 SRAM 的长度，生成局部变量的栈空间包含在 ZI-data 区的范围。